CLIMATE MONITORING SYSTEM

Fase 4

Documentación técnica y guía de operación

OAuth / OIDC · VPS · Arduino Uno
DHT11 · Puente C# · Dashboard en tiempo real

4 de noviembre de 2026

Edición consolidada con guías completas de cambio de UUID y puesta en marcha desde otra PC.

Estado transparente: pruebas locales verificadas, operación VPS reportada por el usuario y pendientes externos diferenciados. Sin credenciales reales.

Contenido

1. Resumen de la entrega . . . 5

2. Estado comprobado y límites de evidencia . . . 6

3. Alcance y continuidad de Fase 3 . . . 7

4. Arquitectura de extremo a extremo . . . 8

5. OAuth 2.0, OIDC y sesión . . . 9

5.1 Proveedores implementados . . . 9

5.2 Flujo seguro . . . 9

5.3 Cuentas y permisos . . . 9

5.4 Contratos OAuth y rutas . . . 10

6. Configuración OAuth local y VPS . . . 11

7. Arduino Uno: dos firmwares diferentes . . . 12

7.1 Cableado de referencia . . . 12

7.2 Protocolo serial v1 . . . 12

8. Compilar y cargar Arduino . . . 13

9. Puente C# .NET 10 . . . 14

9.1 Archivos de operación . . . 14

9.2 Configuración conceptual . . . 14

10. Buffer, reintentos y límites . . . 15

11. API de telemetría y persistencia . . . 16

11.1 Idempotencia y auditoría . . . 16

12. Alertas y entrega de eventos . . . 17

13. Dashboard, históricos y comunicación . . . 18

14. Despliegue y actualización en el VPS . . . 19

14.1 Publicación y pull . . . 19

14.2 Compose y conservación de datos . . . 19
14.3 Diferencia entre configuración e imagen . . . 19
15. Diagnósticos reales resueltos durante la fase . . . 20
16. Pruebas, resultados y evidencia . . . 21
17. Seguridad y gestión de configuración . . . 22
18. Requisitos oficiales pendientes y límites de cierre . . . 23
19. Organización del código y entrega . . . 27
20. Operación diaria y recuperación . . . 28
21. Referencias técnicas . . . 29
Anexo A. Cambiar UUID y configurar la asociación en el VPS . . . 30
Qué identificador cambia y dónde . . . 30
Paso 1 — Crear o elegir el sensor en la web . . . 30
Paso 2 — Detener el puente actual, en Windows . . . 30
Paso 3 — Actualizar la asociación local, en PowerShell . . . 30
Paso 4 — Copiar solo los cinco parámetros Arduino . . . 31
Paso 5 — Guardar los parámetros en Bash del VPS . . . 31
Paso 6 — Comprobar el overlay en el VPS . . . 32
Paso 7 — Aplicar el cambio a Monitoring, en Bash del VPS . . . 32
Paso 8 — Verificar lo recibido por el contenedor . . . 32
Paso 9 — Iniciar el puente en Windows . . . 33
Paso 10 — Comprobar el dashboard . . . 33
Errores y cómo retomar . . . 33
Anexo B. Arduino y puente desde una PC Windows nueva . . . 34
Paso 1 — Detener el emisor de la PC anterior . . . 34
Paso 2 — Copiar los archivos necesarios . . . 34
Paso 3 — Instalar SDK .NET 10 . . . 34

Paso 4 — Conectar el Uno y encontrar el puerto . . . 35
Paso 5 — Revisar appsettings.json sin imprimir el token . . . 35
Paso 6 — Compilar y ejecutar el puente . . . 35
Paso 7 opcional — Instalar IDE para cambiar o cargar el firmware . . . 36
¿Cuándo debo volver a configurar el VPS? . . . 37
Diagnóstico y recuperación . . . 37
Anexo C. Matriz completa de requisitos . . . 38
Anexo D. Firmware DHT11 físico . . . 52
Anexo E. Firmware simulado y referencia LM35 . . . 53
Anexo F. Regeneración y evidencias . . . 54

Climate Monitoring System. Edición consolidada: 4 de noviembre de 2026, zona America/Mexico_City. Incluye los cambios posteriores al cierre local: despliegue Docker en VPS, GitHub real, Arduino Uno, puente HTTPS y adaptación DHT11.

1. Resumen de la entrega

La fase incorpora autenticación externa y adquisición de temperatura por Arduino en la arquitectura de microservicios existente. Auth centraliza Google OIDC y GitHub OAuth; Monitoring recibe telemetría autorizada por dispositivo, persiste lecturas y reutiliza el motor de alertas y el hub SignalR. Angular muestra temperatura actual e histórica por sensor, comunidad, fuente y última comunicación.

El Arduino Uno no realiza solicitudes HTTPS directamente: produce JSON por USB. Una aplicación .NET en la computadora conectada a la placa lee el puerto y publica al dominio del VPS. La PC debe permanecer encendida y conectada para transmitir.

Este documento sustituye las afirmaciones de estado de los informes iniciales que todavía consideraban todo despliegue VPS y prueba física pendientes. La implementación de código y los resultados automatizados se distinguen de las comprobaciones aportadas por el usuario. No se declara Kubernetes desplegado.

2. Estado comprobado y límites de evidencia

Elemento	Estado al cierre de esta edición	Base de la afirmación
Auth, Gateway, Monitoring y Angular	Código implementado y compilado	Código fuente y pruebas registradas
Google OIDC	Integrado y probado con proveedor simulado	Pruebas OIDC; sin confirmación de login Google real
GitHub OAuth	Login real confirmado por el usuario	Conversación y diagnóstico de su VPS; no prueba remota autónoma de esta generación
VPS Docker Compose	Usuario mostró servicios saludables y endpoints del dominio	Salidas aportadas; no es un inventario remoto consultado al generar el PDF
Arduino Uno	Detectado COM3; compilación real; carga del simulador confirmada por usuario	Arduino CLI y salida de upload aportada
Telemetría al dominio	Puente mostró HTTP200 después de actualizar Monitoring	Salida del usuario; confirma aceptación de la lectura por el backend
DHT11 físico	Adaptador compilado y protocolo compatible	Flash 5626 bytes, RAM 243 bytes; falta confirmar carga/medición física DHT11
Humedad DHT11	Campo auxiliar en JSON serial	No existe ingestión independiente de humedad en el puente actual
Kubernetes/K3s	No ejecutado	Solo manifiestos/documentación existentes

No se atribuye una calibración o medición térmica real a una prueba con datos simulados. Una respuesta HTTP200 tampoco es, por sí sola, evidencia visual del dashboard productivo; la publicación/actualización se comprobó en integración local y debe verificarse en el navegador del dominio.

3. Alcance y continuidad de Fase 3

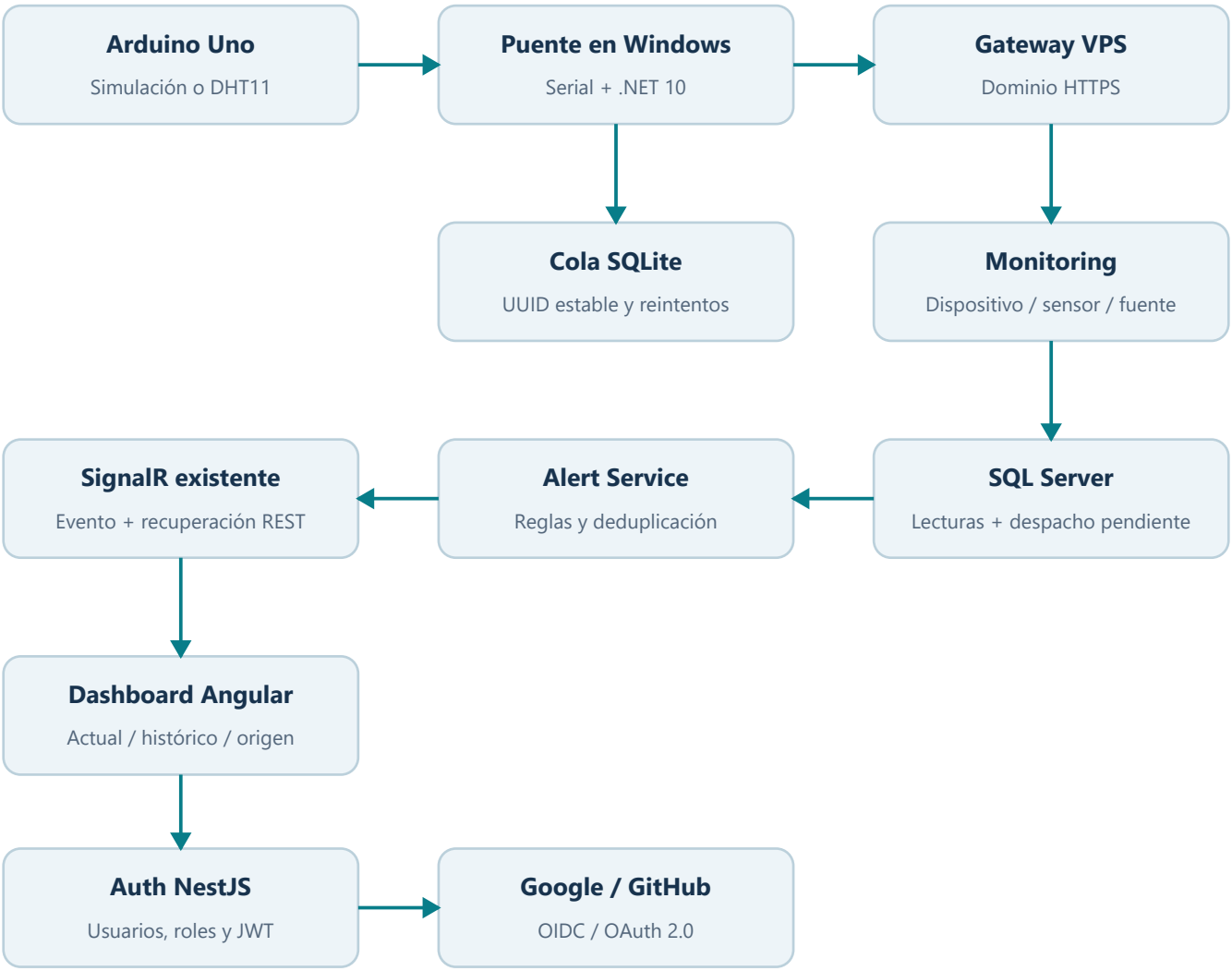
Se conservaron las entidades Sensor y Comunidad, los contratos públicos del Gateway, los JWT/roles, la simulación del servidor, los reportes, la auditoría y los servicios propietarios de sus datos. No se creó otro microservicio ni un hub exclusivo para Arduino.

El alcance inicial excluía operaciones VPS. Posteriormente el usuario solicitó actualizar su despliegue existente y ejecutó los comandos documentados de Docker Compose, dominio HTTPS y configuración OAuth. No se instalaron ni aplicaron clústeres Kubernetes desde esta fase.

Servicio	Tecnología / persistencia de la implementación	Responsabilidad
Auth	NestJS / PostgreSQL	Usuarios, proveedores, roles, sesión y JWT
Gateway	ASP.NET Core / YARP	Rutas públicas, validación JWT, proxy y CORS
Sensor	.NET / SQL Server	Catálogo, tipo, unidad, estado y comunidad
Community	.NET / PostgreSQL	Propiedad de comunidades
Monitoring	.NET / SQL Server	Ingestión, lecturas, históricos, simulación y SignalR
Alert	Java Spring Boot / PostgreSQL	Reglas, niveles y deduplicación de evaluaciones
Event	.NET / SQL Server	Eventos climáticos existentes
Audit	Python FastAPI / MongoDB	Auditoría asíncrona existente
Report	Python FastAPI / MongoDB	Reportes existentes
Web	Angular / Chart.js	Login, dashboard, administración y visualización
ArduinoBridge	.NET 10 / SQLite local	Serial, autorización de dispositivo, cola y HTTPS

Versiones verificadas durante la implementación: .NET 10, openid-client 6.8.8, Arduino IDE 2.3.10, Arduino CLI 1.5.1, Arduino AVR Boards 1.8.8, DHT sensor library 1.4.7 y Adafruit Unified Sensor 1.1.15. El frontend usa Angular 22.2.3 según el proyecto. El VPS documentado utiliza MongoDB 7 por compatibilidad; no montar allí los volúmenes de MongoDB 8.

4. Arquitectura de extremo a extremo



En telemetría, el Gateway transforma `/api/monitoring/telemetry` en `/api/v1/monitoring/telemetry`. Monitoring deriva comunidad/tipo/unidad del catálogo de sensores. El identificador del sensor se configura en el puente, no se obtiene del nombre del sketch.

El flujo de identidad es independiente de la credencial IoT. El puente no usa la cuenta administradora, el token de una persona ni el Client Secret de GitHub.

5. OAuth 2.0, OIDC y sesión

5.1 Proveedores implementados

Google usa OIDC discovery con issuer `https://accounts.google.com`. openid-client valida nonce, claims y firma del ID token con verificación de no repudio habilitada. GitHub usa Authorization Code OAuth2, PKCE y su API de perfil/correos; no se le atribuye emisión de ID tokens para login de usuarios. Su issuer configurado es `https://github.com/login/oauth`.

El usuario registra su propia OAuth App y obtiene Client ID/Secret. El código no inventa credenciales ni necesita Kong o Keycloak para este diseño. Auth sigue siendo el punto central de identidad y el Gateway conserva la función de proxy y protección de APIs.

5.2 Flujo seguro

1. Angular genera un verifier aleatorio, conserva su valor en sessionStorage y envía su challenge a Auth.
2. Auth genera state, verifier PKCE del proveedor, nonce OIDC y vínculo al navegador mediante cookie HttpOnly.
3. El estado de cinco minutos se guarda en PostgreSQL; el state y el vínculo de cookie se almacenan como hashes.
4. El proveedor retorna al callback del mismo dominio HTTPS.
5. Auth verifica cookie/state y consume el flujo. Valida la respuesta del proveedor e identidad/correo verificado.
6. Auth produce un código propio de uso único con vigencia de 60 s y challenge del navegador. No coloca JWT en query string.
7. Angular limpia el fragmento y canjea ese código con su verifier; Auth emite el JWT del sistema.
8. Guards/interceptor y los servicios continúan aplicando roles, firma, issuer, audience y expiración.

La cookie usa Secure en callbacks HTTPS, SameSite=Lax, HttpOnly y Path=/. No debe quitarse la validación de cookie/state para resolver un error de despliegue. Reintentar desde `/login`: recargar un callback consumido no reinicia el flujo.

5.3 Cuentas y permisos

Se conservan Administrator, Operator y Viewer, equivalentes funcionales a ADMIN, OPERATOR y VIEWER. Los nuevos usuarios externos reciben Viewer, nunca administración automática. La contraseña local de una cuenta exclusivamente externa no es utilizable para login local.

No se fusionan cuentas solo porque compartan correo. Un correo que pertenece a una cuenta local requiere vinculación autenticada explícita. Existe endpoint backend de vinculación con AuthGuard, pero no se documenta una pantalla de vinculación que no se haya implementado. Cuenta inactiva no recibe JWT en el canje.

Logout conserva la revocación implementada en Auth y limpia sesión/conexión del cliente. La revocación inmediata global de JWT en cada microservicio no se añadió; los validadores locales siguen respetando la expiración configurada.

5.4 Contratos OAuth y rutas

Ruta pública Gateway	Uso y protección
GET /api/auth/oauth/{provider}/start?challenge=...	Inicio público con challenge válido
GET /api/auth/oauth/{provider}/callback	Público respecto a JWT humano; valida state/cookie/proveedor
POST /api/auth/oauth/exchange	Código propio de un uso y verifier del navegador
POST /api/auth/oauth/{provider}/link	Requiere identidad JWT mediante AuthGuard

La ruta YARP `oauth-public` tiene prioridad-20 y política Anonymous. Eso no elimina el guard del endpoint link. No abrir todo el backend ni eliminar la política de las rutas administrativas.

6. Configuración OAuth local y VPS

La configuración privada productiva usa estos nombres y ejemplos sin valores reales:

```
OAUTH_FRONTEND_URL=https://analisissistemas2026proyecto.xyz
OAUTH_GITHUB_CLIENT_ID=CONFIGURAR_CLIENT_ID
OAUTH_GITHUB_CLIENT_SECRET=CONFIGURAR_SECRET_PRIVADO
OAUTH_GITHUB_CALLBACK_URL=https://analisissistemas2026proyecto.xyz/api/auth/oauth/github/callback
OAUTH_GOOGLE_CLIENT_ID=CONFIGURAR_CLIENT_ID
OAUTH_GOOGLE_CLIENT_SECRET=CONFIGURAR_SECRET_PRIVADO
OAUTH_GOOGLE_CALLBACK_URL=https://analisissistemas2026proyecto.xyz/api/auth/oauth/google/callback
```

Google y GitHub se configuran por separado: uno puede funcionar aunque falten las credenciales del otro. Registrar el callback exacto en cada proveedor. Para desarrollo se utiliza localhost; para VPS se mantiene el dominio HTTPS y OAUTH_ALLOW_LOCAL_HTTP=false. Es preferible separar las aplicaciones OAuth de desarrollo y producción.

Un archivo `.env` sirve como fuente para interpolación de Compose, pero no inyecta automáticamente cada variable al contenedor. El servicio identity-service necesita su mapping environment en el overlay OAuth. No copiar esos secretos a Angular ni al puente.

El frontend genera runtime-config.json desde GATEWAY_PUBLIC_URL. En producción:

```
{"production":true,"apiUrl":"https://analisissistemas2026proyecto.xyz","realtime":{"enabled":true,"hubUrl":"
  https://analisissistemas2026proyecto.xyz/hubs/monitoring"}}
```

GATEWAY_PUBLIC_URL de Compose y Backend.BaseUrl del puente son configuraciones diferentes. Cambiar una no modifica la otra. No mezclar el overlay local docker-compose.phase4.yml, que fija localhost, con el despliegue productivo.

7. Arduino Uno: dos firmwares diferentes

Archivo	Entrada	Origen emitido
hardware/arduino/temperature-sensor/temperature-sensor.ino	Onda de temperatura sin sensor, SIMULATE_SENSOR=1	SIMULATED
Mismo archivo con SIMULATE_SENSOR=0	Ejemplo de referencia LM35 en A0, ADC 5 V / 10 bits	ARDUINO
backend/lecturas-sensor/lecturas-sensor.ino	DHT11 físico con DATA en pin digital2	ARDUINO

Se carga un sketch a la vez: el último reemplaza el anterior. El Arduino conserva el programa al desconectarse de la PC. No dispone de Wi-Fi/Ethernet por este diseño y no conoce el UUID del backend.

El firmware simulado produce entre20y40°C cada 2 s; el DHT11 toma muestras cada 3 s. Ambos usan millis para evitar delay de muestreo en el loop y emiten un JSON terminado en newline. La librería DHT requiere su temporización de lectura interna; no se declara que toda operación de sensor sea no bloqueante.

7.1 Cableado de referencia

En el DHT11 usar el pin de datos del módulo hacia D2, tierra común y alimentación conforme a la etiqueta/especificación del módulo. No asumir orientación de pines por color o forma. Un sensor suelto de cuatro pines puede necesitar pull-up según fabricante; un módulo puede incorporarlo. La referencia LM35 usa salida analógica a A0 y conversión10mV/°C, ajustando la referencia ADC al voltaje real.

El firmware no es universal para DHT22, DS18B20 u otras placas: adaptar lectura, librería y cableado conservando el contrato serial. No se afirma calibración física por una compilación exitosa.

7.2 Protocolo serial v1

```
{
  "protocolVersion": 1,
  "deviceId": "arduino-temp-001",
  "sensorType": "TEMPERATURE",
  "value": 27.5,
  "unit": "C",
  "source": "ARDUINO",
  "sequence": 125
}
```

Sin reloj confiable en el Uno se omite timestamp y el puente asigna UTC de recepción. No se generan fechas ficticias. Sequence es contador de muestras; el UUID estable para reintentos HTTP se genera y persiste en el puente antes de enviar.

El DHT11 añade sensor y humidity como campos auxiliares. Actualmente se ignoran para ingestión: el dashboard recibe temperatura, no una segunda lectura de humedad. Los errores DHT11_READ_FAILED se registran/rechazan como errores de sensor, no como temperatura0.

8. Compilar y cargar Arduino

En Windows se preparó Arduino IDE 2.3.10, CLI1.5.1 y AVR Boards 1.8.8. El Uno fue identificado en COM3 por Arduino CLI. Otro equipo puede asignar otro puerto. DHT requiere DHT sensor library de Adafruit y Adafruit Unified Sensor; referencias al final.

Desde raíz Windows, con puente y Monitor Serial cerrados:

```
# Verificar simulador sin cargar
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Arduino-Uno.ps1
# Cargar simulador
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Arduino-Uno.ps1 -Upload -PortName COM3
# Verificar DHT11 sin cargar
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Arduino-Uno.ps1 -SketchPath `
  backend/lecturas-sensor
# Cargar DHT11
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Arduino-Uno.ps1 -SketchPath `
  backend/lecturas-sensor -Upload -PortName COM3
```

El comando sin -SketchPath selecciona el simulador anterior. No ejecutar las dos cargas si se quiere conservar una sola modalidad. El Monitor Serial se configura a 9600 baudios y se cierra antes de abrir el puente. No se necesitan drivers adicionales si Windows ya reconoce el puerto; un conversor USB distinto exige revisar su driver particular.

9. Puente C# .NET 10

La aplicación Worker/Console utiliza DI, IOptions validadas, ISerialDataSource, System.IO.Ports, BackgroundService, IHttpClientFactory, ILogger y CancellationToken. Program.cs se limita a configuración y composición de servicios.

AcquisitionWorker abre el puerto o lo detecta si existe exactamente uno, lee frames acotados a 4096 caracteres, valida JSON/dispositivo/unidad/rango y asigna sensor/UTC/UUID. SerialDataSource también permite generar muestras sin hardware mediante -Simulate.

TransmissionWorker publica lo persistido usando X-Device-Token. No requiere usuario administrador. Dos PCs no deben emitir simultáneamente con la misma identidad. La protección del lanzador evita duplicar instancias del mismo lanzador en una sesión Windows; no es un bloqueo distribuido entre PCs.

9.1 Archivos de operación

Archivo	Contenido y tratamiento
tools/Climate.ArduinoBridge/appsettings.json	Puerto, dominio, UUID y límites; configuración privada del equipo
tools/Climate.ArduinoBridge/device-token.local	Token original; no versionar ni compartir públicamente
backend/.env.phase3.local	Parámetros del entorno; en VPS es privado y conserva secretos de sus servicios
backend/docker-compose.arduino.vps.yml	Mapping de ARDUINO_* al servicio Monitoring; no contiene secretos reales
Archivo SQLite configurado	Pendientes del puente; conservar al reiniciar y no mezclar con otro dispositivo

El archivo appsettings.example.json es un ejemplo local; no copiarlo sin ajustar el dominio HTTPS. El token se inyecta por el lanzador y no forma parte del JSON compartido del ejemplo.

9.2 Configuración conceptual

```
{
  "Arduino": { "PortName": "COM3", "BaudRate": 9600, "ReconnectSeconds": 5, "Simulate": false },
  "Backend": { "BaseUrl": "https://analisissistemas2026proyecto.xyz", "TelemetryPath":
  "/api/monitoring/telemetry", "TimeoutSeconds": 10, "AllowLocalHttp": false },
  "Device": { "DeviceId": "arduino-temp-001", "SensorId": "UUID_REAL_DEL_SENSOR_VPS" },
  "Buffer": { "Path": "arduino-vps-buffer.db", "MaxReadings": 10000, "RetentionHours": 168 }
}
```

Este JSON usa un marcador de UUID, no un valor ejecutable. No se cambia el UUID al azar: debe existir como sensor activo de temperatura en el VPS.

10. Buffer, reintentos y límites

SQLite usa WAL y FIFO por rowid. Cada payload se guarda antes de publicar; el UUID no cambia en reintentos HTTP. Se elimina después de una respuesta exitosa. Backend caído, fallo de red, 429 y 5xx provocan backoff de 2 a 64 s.

Condición	Política implementada
HTTP 200 / 2xx	Eliminar pendiente confirmado y continuar
HTTP 401 / 403	Detener envío, conservar pendientes y pedir corregir autorización; reiniciar después
HTTP 400 / 404 / 409 y otros 4xx permanentes distintos de 429	Registrar rechazo y eliminar ese pendiente; no reintentar para siempre
Retención superada	Registrar vencimiento y descartar al despachar
Límite de filas alcanzado	Diagnóstico explícito de buffer lleno y reconexión/adquisición interrumpida

Por defecto se admiten 10 000 lecturas pendientes y 168 h. El límite es de filas, no una cuota exacta de bytes SQLite/WAL. La falta de entrega no es una garantía ilimitada: muestras fuera de los límites pueden perderse. Un 404 de despliegue se debe corregir con el puente detenido porque se considera rechazo permanente.

Al cambiar de UUID o destino, usar una cola independiente y conservar la anterior para recuperación controlada. No editar UUID/source dentro de payloads ya pendientes para forzar su aceptación. Un nuevo UUID no mueve los históricos anteriores.

11. API de telemetría y persistencia

Ruta externa: POST `https://analisissistemas2026proyecto.xyz/api/monitoring/telemetry`. Interna: POST `/api/v1/monitoring/telemetry`. El Gateway permite llegar a la ruta sin JWT humano y Monitoring exige credencial de dispositivo.

```
{
  "readingId": "UUID_ESTABLE_DE_LECTURA",
  "sensorId": "UUID_EXISTENTE_DEL_SENSOR",
  "deviceId": "arduino-temp-001",
  "value": 27.5,
  "unit": "C",
  "measuredAt": "2026-11-04T20:00:00Z",
  "source": "ARDUINO"
}
```

Es un ejemplo de contrato; fecha y UUID reales los construye el puente. Header privado: X-Device-Token. El servidor compara SHA256 del token en tiempo constante y limita su uso al dispositivo/sensor/source configurados. Un JWT humano por sí solo no autoriza ingestión.

Se comprueban UUID válido, dispositivo activo/registrado, sensor existente y activo, Temperature, unidadC/°C, decimal con máximo4decimales, rango configurable por defecto-55..150°C y fecha entre7días anteriores y 5 minutos futuros. La propiedad value es obligatoria; JSON incompleto no se convierte silenciosamente en cero.

SensorReading conserva el modelo existente y agrega Source, DeviceId y TelemetryDispatched mediante migración20261029000100_Phase4Telemetry. Históricos anteriores usan SIMULATED. El sensor/comunidad no se duplican. La asociación vive en Telemetry.Devices y la comunidad procede del catálogo.

11.1 Idempotencia y auditoría

ReadingId es clave primaria. La búsqueda/inserción usa bloqueos SQL UPDLOCK/HOLDLOCK y reutiliza la transacción abierta por el filtro de auditoría. Si no hay transacción, el controlador crea una serializable. Repetir UUID con los mismos datos devuelve la lectura; cambiar sensor/dispositivo/valor/source/fecha produce 409. Conflictos SQL transitorios pueden provocar 5xx y reintento del puente.

Se corrigió una transacción anidada detectada en la integración inicial. No se eliminó el filtro de auditoría para evitar el error.

12. Alertas y entrega de eventos

Una lectura aceptada queda persistida antes de confirmar HTTP. Un worker consulta hasta 100 lecturas pendientes, ordenadas por fecha, cada 2 s y ejecuta el evaluador de Alert existente. El servicio Java deduplica por readingId para no repetir alertas por reintentos de la misma lectura.

Después se publica SensorReadingUpdated en el hub actual y se marca el despacho. Un fallo conserva el estado pendiente. La entrega es al menos una vez: si el proceso cae después de publicar y antes de marcar, SignalR puede repetirse. Los consumidores hacen upsert/recuperación REST. No se promete que un navegador desconectado reciba cada evento histórico individual.

Se preservan los niveles Verde, Amarillo, Naranja y Rojo y las reglas configurables del sistema. El formato de una lectura física y simulada recorre el mismo pipeline de evaluación.

13. Dashboard, históricos y comunicación

La gráfica de temperatura utiliza SensorChart existente e incorpora un selector de sensor y comunidad. Muestra valor/°C, último timestamp, origen y comunicación reciente/obsoleta. El histórico viene del backend, agrupado por bucket y source. ARDUINO y SIMULATED tienen estilos distinguibles; Angular no inventa temperaturas.

SignalR dispara sincronización del panel y REST recupera estado cada 5 s; la infraestructura existente también reconecta. Después de 30 s sin lectura, por defecto, se muestran datos obsoletos. Ese umbral se puede configurar mediante input del componente; no se deduce conexión únicamente porque el sensor exista en la base.

Crear un sensor en el catálogo no lo asocia a Arduino. Mientras su UUID no aparezca en Telemetry.Devices, el simulador del servidor puede generarle valores. Una vez asociado, queda excluido del simulador y de la inserción manual, incluso si se inactiva el dispositivo, para no aparentar lecturas físicas frescas.

Si el puente apunta al dominio, el dashboard local puede quedar obsoleto: son bases y destinos diferentes. No interpretar el cambio de PC o el prefijo del nombre del sensor como una migración de datos.

14. Despliegue y actualización en el VPS

Proyecto Docker existente: climate-phase3-vps. Backend documentado: `/opt/climate/climate_backend`. Frontend: `/opt/climate/Climate_Frontend`, enlazado desde `/opt/climate/frontend/climate-monitoring-web`. Proxy público existente: Nginx; Gateway enlazado a 127.0.0.1:18080 en el override Hostinger. Estas rutas describen la instalación reportada, no una instalación universal.

14.1 Publicación y pull

Solo se descargan cambios que ya se publicaron en GitHub. `.env.phase3.local`, configuración privada del puente y token no deben versionarse. Revisar `git status --short` antes de un pull; usar `git pull --ff-only` y resolver divergencias explícitamente, sin borrar cambios por rutina.

```
cd /opt/climate/climate_backend
git status --short
git pull --ff-only
git -C ../frontend/climate-monitoring-web status --short
git -C ../frontend/climate-monitoring-web pull --ff-only
```

14.2 Compose y conservación de datos

Conservar el mismo project-name y los mismos volúmenes. El overlay MongoVPS mantiene MongoDB 7 y sus volúmenes separados. El overlay de dominio fija URL pública del frontend; el OAuth inyecta parámetros a identity-service; el Arduino inyecta ARDUINO_* a monitoring-service. No se debe perder un overlay al recrear servicios con otra lista Compose.

Para cambiar un servicio, recuperar sus archivos reales desde sus labels. Ejemplo completo para Monitoring:

```
cd /opt/climate/climate_backend
CONFIG_FILES=$(docker inspect --format \
  '{{ index .Config.Labels "com.docker.compose.project.config_files" }}' \
  climate-phase3-vps-monitoring-service-1)
IFS=',' read -r -a FILES <<< "$CONFIG_FILES"
COMPOSE=(docker compose --project-name climate-phase3-vps --env-file .env.phase3.local)
for FILE in "${FILES[@]}"; do COMPOSE+=(-f "$FILE"); done
"${COMPOSE[@]}" config --quiet
```

Si CONFIG_FILES sale vacío, detenerse y verificar el nombre del contenedor. Si se trata de un overlay nuevo, agregarlo al final de COMPOSE como en las guías anexas. Cerrar la terminal pierde esa variable, pero no borra los archivos ni datos.

14.3 Diferencia entre configuración e imagen

Cambiar environment requiere recrear el contenedor, no recompilar. Cambiar C#/TypeScript/frontend requiere construir la imagen antes del up. `--no-build` solo usa la imagen que ya existe; no incorpora un endpoint nuevo.

```
# Solo cambia UUID/token/source, con la imagen actual ya validada
"${COMPOSE[@]}" up -d --no-deps --no-build --force-recreate --wait --wait-timeout 180 monitoring-service
# Cambia codigo de Monitoring
"${COMPOSE[@]}" build monitoring-service &&
"${COMPOSE[@]}" up -d --no-deps --no-build --force-recreate --wait --wait-timeout 180 monitoring-service
```

No ejecutar down-v, borrar volúmenes ni seeds completos para aplicar un cambio de UUID. Build antes de up permite conservar el contenedor anterior si la construcción falla. El reinicio puede interrumpir brevemente el servicio; no se documenta despliegue sin interrupción garantizado.

15. Diagnósticos reales resueltos durante la fase

Síntoma observado	Diagnóstico	Corrección y comprobación
Healthy pero conexión Windows cerrada	Health interno no prueba el puerto publicado	Recrear solo Gateway/frontend; arranque local verifica endpoint público antes del seed
Callback con issuer GitHub y401	Metadata usaba https://github.com en vez de /login/oauth	Corregir issuer y reconstruir Auth; prueba reproduce y rechaza issuer ajeno
Callback401 vacío, WWW-Authenticate:Bearer	Gateway antiguo exigía JWT antes de Auth	Reconstruir Gateway con oauth-public; curl sin sesión llega a Auth
SESSION_MISSING tras prueba curl	Curl no lleva cookie/state de un flujo	Resultado esperado sin login; iniciar un flujo nuevo en navegador
runtime-config apunta localhost	Frontend productivo usa URL local	Inyectar dominio en frontend y recrearlo; runtime-config confirma HTTPS
PROVIDER_NOT_CONFIGURED	Faltan variables dentro de Auth aunque estén en .env	Agregar mapping OAuth al overlay y recrear solo Auth
Sensor local obsoleto / VPS genera solo	Bases distintas y UUID VPS sin asociación IoT	Configurar UUID real del VPS, overlay y reiniciar Monitoring
POST telemetría404	Imagen Monitoring anterior sin controlador nuevo	Build+up de Monitoring; prueba incompleta cambia a 400 y puente a 200
Error de transacción anidada	Filtro audit ya abrió transacción	Reutilizar transacción y bloqueos de idempotencia

Diagnóstico de Auth usa etiquetas fijas en logs y headers X-OAuth-Error/X-OAuth-Reference, sin códigos OAuth, cookies, tokens ni secretos. ACCOUNT_LINK_REQUIRED significa que el correo pertenece a una cuenta que requiere vinculación; no se evita eliminando la validación. Los errores de certificado se corrigen con confianza válida, no desactivando TLS.

16. Pruebas, resultados y evidencia

Últimas ejecuciones automatizadas registradas a lo largo de la fase, no una nueva ejecución masiva al generar este PDF:

Grupo	Último resultado registrado	Cobertura destacada
Auth	14 aprobadas	JWT, contraseñas, callback GitHub/issuer, Google simulado, nonce/firma/claims, diagnóstico sin secretos
Monitoring	18 aprobadas	Regresión, sensores inexistentes/inactivos, exclusión del simulador
ArduinoBridge	13 aprobadas	JSON/campos requeridos, rango, dispositivo, TLS, SQLite/reinicio/UUID/límite, reconexión,401,503→200
Angular	57 aprobadas	Contratos/guards y panel que actualiza histórico/valor/obsolescencia
Integración local	15 comprobaciones aprobadas	HTTP, SQL, UUID idempotente, alertas, SignalR, puente simulado
Edge local	Tres auditorías sin violaciones detectadas	Login, dashboard escritorio/móvil, sin errores JS y responsive

Son 102 pruebas automatizadas en los últimos resultados registrados, más las comprobaciones de integración y navegador. La primera entrega tenía99; las pruebas de issuer/diagnóstico añadidas posteriormente explican la diferencia.

Artefactos conservados: docs/validation/phase4-integration.json, phase4-browser.json, capturas del dashboard; scripts tools/phase4-integration.mjs y phase4-browser.mjs. Los scripts son de entorno local y pueden registrar/actualizar la asociación demo local. No ejecutarlos contra producción como si fueran una prueba de solo lectura.

Compilación Arduino simulador: 3920 bytes flash, 204 bytes RAM; DHT11: 5626 bytes flash, 243 bytes RAM. El usuario aportó carga exitosa del simulador Uno y respuesta HTTP 200 de la telemetría al VPS. DHT11 tiene compilación real pero no una lectura física confirmada en esta edición. No se incorpora un certificado de calibración inexistente.

Para repetir pruebas desde raíz Windows:

```
node backend/src/Services/AuthService/node_modules/typescript/bin/tsc -p `
  backend/src/Services/AuthService/tsconfig.json
node --test backend/src/Services/AuthService/tests/*.test.cjs
dotnet test backend/tests/Climate.Monitoring.Tests/Climate.Monitoring.Tests.csproj --no-restore
dotnet test tools/Climate.ArduinoBridge.Tests/Climate.ArduinoBridge.Tests.csproj
```

Angular se comprueba desde frontend/climate-monitoring-web con el Node compatible tools/node/node.exe:

```
../../tools/node/node.exe node_modules/@angular/cli/bin/ng.js test --watch=false
../../tools/node/node.exe node_modules/@angular/cli/bin/ng.js build
../../tools/node/node.exe node_modules/@angular/cli/bin/ng.js lint
```

17. Seguridad y gestión de configuración

Este PDF no contiene credenciales reales, valores de .env, token IoT, Client Secrets, contraseñas, JWT ni cookies. Los ejemplos usan marcadores. Los archivos privados no se copian automáticamente a otra PC por Git.

Secretos compartidos durante soporte deben rotarse. La rotación de contraseñas DB debe coordinarse con el servicio/base que las usa; cambiar solo `.env` no cambia la contraseña persistida de una base existente. No utilizar esta documentación como sustituto de un plan de backup/restauración de producción.

Mantener HTTPS en el VPS, CORS con dominio explícito y token IoT por dispositivo. El hash del token se registra en servidor, el original se protege en PC. Para otro dispositivo independiente usar sensorId/deviceId/credencial propios y ampliar Telemetry.Devices; el overlay entregado configura un dispositivo en posición0.

18. Requisitos oficiales pendientes y límites de cierre

La implementación OAuth/IOT y la operación Docker confirmada no cumplen por sí mismas la finalización oficial de Kubernetes/K3s. La matriz adjunta usa IDs RF/RNF oficiales y etiquetas locales K8S/ARD ligadas a secciones. No se atribuye prueba productiva a una prueba local.

RF-OAUTH-03: Google real — Código y proveedor simulado

Requisito oficial / adicional	RF-OAUTH-03: Google real
Estado	Código y proveedor simulado
Motivo pendiente	No se confirmó login con credenciales reales
Implementación reutilizable	OAuthService, Angular y tests OIDC
Dependencia externa	OAuth App Google y cuentas de prueba
Acción del responsable	Registrar callbacks y probar flujo real
Cómo verificar	Login, rol Viewer y rechazo de respuestas inválidas
Riesgo	Restricciones reales del proveedor sin validar

Arduino: medición DHT11 — Adaptador compilado

Requisito oficial / adicional	Arduino: medición DHT11
Estado	Adaptador compilado
Motivo pendiente	Falta confirmación de carga y medición física DHT11
Implementación reutilizable	Sketch, puente y API
Dependencia externa	Sensor, cableado y referencia térmica
Acción del responsable	Revisar conexiones y comparar medición
Cómo verificar	JSON serial, HTTP200 y valor coherente en dashboard
Riesgo	Lecturas inválidas o desviación sin detectar

§10: clúster K3s — No instalado

Requisito oficial / adicional	§10: clúster K3s
Estado	No instalado
Motivo pendiente	Ejecución Kubernetes excluida
Implementación reutilizable	backend/k8s
Dependencia externa	VPS autorizado y recursos
Acción del responsable	Dimensionar y preparar clúster en tarea posterior
Cómo verificar	Nodos Ready y reinicio controlado
Riesgo	Sin orquestación Kubernetes demostrada

§11: Pods — No validados en clúster

Requisito oficial / adicional	§11: Pods
Estado	No validados en clúster
Motivo pendiente	No se aplicaron manifiestos
Implementación reutilizable	Deployments existentes
Dependencia externa	Clúster operativo
Acción del responsable	Desplegar imágenes revisadas
Cómo verificar	Pods Ready, logs y reinicios
Riesgo	Fallos reales de arranque desconocidos

§11: Deployments — No aplicados

Requisito oficial / adicional	§11: Deployments
Estado	No aplicados
Motivo pendiente	Despliegue K3s no ejecutado
Implementación reutilizable	Manifiestos de Fase3
Dependencia externa	Registry y clúster
Acción del responsable	Preparar rollout y rollback
Cómo verificar	rollout status y recuperación de versión
Riesgo	Sin recuperación productiva demostrada

§12: Services internos — No comprobados en K8s

Requisito oficial / adicional	§12: Services internos
Estado	No comprobados en K8s
Motivo pendiente	Compose no prueba DNS/selectores K8s
Implementación reutilizable	Contratos/Services existentes
Dependencia externa	DNS de clúster
Acción del responsable	Revisar selectores y puertos
Cómo verificar	Consultas internas desde Pod autorizado
Riesgo	APIs internas inaccesibles

§12: Ingress — No implementado en K3s

Requisito oficial / adicional	§12: Ingress
Estado	No implementado en K3s
Motivo pendiente	Nginx del VPS no es Ingress validado
Implementación reutilizable	Gateway y rutas del dominio
Dependencia externa	Controller e ingressClass
Acción del responsable	Configurar host/rutas y WebSocket
Cómo verificar	REST y SignalR por Ingress
Riesgo	Tiempo real o callbacks podrían fallar

§13: ConfigMaps — Preparación por entorno

Requisito oficial / adicional	§13: ConfigMaps
Estado	Preparación por entorno
Motivo pendiente	No se aplicaron al clúster
Implementación reutilizable	Opciones de servicios
Dependencia externa	Endpoints internos definitivos
Acción del responsable	Inyectar parámetros no sensibles
Cómo verificar	Configuración efectiva y health
Riesgo	URLs/flags productivos incorrectos

RNF-OAUTH-02 / §13: Secrets K8s — Pendiente

Requisito oficial / adicional	RNF-OAUTH-02 / §13: Secrets K8s
Estado	Pendiente
Motivo pendiente	.env productivo no acredita Kubernetes Secrets
Implementación reutilizable	Inyección por entorno
Dependencia externa	Almacén seguro y credenciales
Acción del responsable	Provisionar, limitar acceso y rotar secretos
Cómo verificar	Login/APIs sin exponer valores
Riesgo	Exposición de secretos o indisponibilidad

§14: Persistent Volumes/PVC — No migrados a K3s

Requisito oficial / adicional	§14: Persistent Volumes/PVC
Estado	No migrados a K3s
Motivo pendiente	Persistencia de clúster fuera del alcance ejecutado
Implementación reutilizable	Bases y volúmenes Docker existentes
Dependencia externa	StorageClass y backups
Acción del responsable	Configurar PVC y restauración de prueba
Cómo verificar	Reiniciar Pod y recuperar datos
Riesgo	Pérdida de datos en migración

§14: liveness/readiness — Endpoints y health Docker existentes

Requisito oficial / adicional	§14: liveness/readiness
Estado	Endpoints y health Docker existentes
Motivo pendiente	Falta ejecutar probes en K3s
Implementación reutilizable	/health y /health/live
Dependencia externa	Clúster operativo
Acción del responsable	Configurar probes y fallos controlados
Cómo verificar	Tráfico solo a instancias listas
Riesgo	Tráfico a procesos no disponibles

Objetivo7: escalabilidad horizontal — No medida

Requisito oficial / adicional	Objetivo7: escalabilidad horizontal
Estado	No medida
Motivo pendiente	No se probó despliegue multirréplica
Implementación reutilizable	UUID idempotente y Deployments
Dependencia externa	Recursos, balanceo y backplane SignalR
Acción del responsable	Diseñar distribución antes de escalar Monitoring
Cómo verificar	Eventos recibidos entre réplicas
Riesgo	Clientes pierden actualizaciones entre instancias

Objetivo8: DNS — Dominio operativo reportado

Requisito oficial / adicional	Objetivo8: DNS
Estado	Dominio operativo reportado
Motivo pendiente	Administración y renovación fuera de esta generación
Implementación reutilizable	URLs configurables
Dependencia externa	Propiedad del dominio/proveedor DNS
Acción del responsable	Conservar registros y monitoreo
Cómo verificar	Resolución desde clientes externos
Riesgo	Dominio inaccesible por cambios o vencimiento

Objetivo8 / §9: TLS público — HTTPS operativo reportado

Requisito oficial / adicional	Objetivo8 / §9: TLS público
Estado	HTTPS operativo reportado
Motivo pendiente	No se emitió ni probó renovación aquí
Implementación reutilizable	Nginx y callbacks HTTPS
Dependencia externa	CA, dominio y tarea de renovación
Acción del responsable	Revisar cadena, vigencia y renovación
Cómo verificar	HTTPS válido y renovación controlada
Riesgo	Interrupción al expirar certificado

Referencia oficial Fase1 — URL respondió404

Requisito oficial / adicional	Referencia oficial Fase1
Estado	URL respondió404
Motivo pendiente	Fuente remota no disponible
Implementación reutilizable	Referencias Fases2/3/4 guardadas
Dependencia externa	Documento del responsable académico
Acción del responsable	Solicitar y conservar fuente
Cómo verificar	Contrastar matriz con requisitos originales
Riesgo	Trazabilidad incompleta de Fase1

19. Organización del código y entrega

Ubicación	Cambio principal
backend/src/Services/AuthService/src/oauth.ts	Proveedores, flujos, PKCE, cuentas externas y sesión
backend/src/Services/AuthService/src/oauth-diagnostics.ts	Clasificación segura de errores
backend/src/Gateway/Climate.Gateway/appsettings.json	Rutas OAuth/telemetría públicas específicas
backend/src/Services/MonitoringService	Controlador, opciones, entidad/migración y despacho persistente
frontend/climate-monitoring-web/src/app	Login externo, servicios y gráfica de temperatura
tools/Climate.ArduinoBridge	Worker serial/HTTP, protocolo, opciones y SQLite
tools/Arduino-Uno.ps1	Compilar/cargar Uno con sketch seleccionable
backend/lecturas-sensor/lecturas-sensor.ino	Adaptador DHT11 físico
hardware/arduino/temperature-sensor/temperature-sensor.ino	Simulación y ejemplo LM35
backend/scripts/Configure-Phase4Arduino.ps1	Configurar asociación/token sin duplicar claves
backend/docker-compose.arduino.vps.yml	Inyectar asociación Arduino al VPS
docs/GUIA_ARDUINO_CAMBIAR_SENSOR_UUID.md	Procedimiento completo de cambio de asociación
docs/GUIA_ARDUINO_NUEVA_PC.md	Puesta en marcha en otro Windows

Los overlays privados OAuth/dominio creados en la VPS pueden no formar parte del repositorio. Conservarlos en todas las actualizaciones que afecten sus servicios. Herramientas, .ino y guías fuera del repo backend no se transfieren a otra PC haciendo únicamente pull del backend.

20. Operación diaria y recuperación

Con Arduino ya programado y VPS operativo, solo se conecta USB, se cierra Monitor Serial y se ejecuta el puente:

```
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Climate.ArduinoBridge/Start-ArduinoBridge.ps1 `
  -PortName COM3 -Configuration Release -NoBuild
```

Compilar si cambia código .NET; cargar firmware si cambia el sketch; recrear Monitoring si cambia UUID/token/source; actualizar frontend si cambia su código o URL pública. No hacer todas esas operaciones para cada encendido.

Las dos guías anexas contienen pasos completos sin depender de comandos de una conversación anterior. Al retomar, registrar el paso alcanzado y consultar:

```
Get-Content .\FASE_4_PROGRESO.md
```

La regeneración del documento no instala controladores, no carga firmware ni ejecuta comandos contra el VPS. Archivo fuente y generador quedan incluidos para actualizar el PDF sin repetir la implementación.

21. Referencias técnicas

- Requisito principal: phase_4.md de la raíz y docs/references/phase_4.official.md, con Fases2/3 oficiales conservadas.
- Google OAuth/OIDC: https://developers.google.com/identity/protocols/oauth2/web-server
- Google OIDC: https://developers.google.com/identity/openid-connect/openid-connect
- GitHub OAuth: https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps
- Issuer GitHub: https://docs.github.com/en/apps/github-authentication-discovery-endpoints
- Cliente: https://github.com/panva/openid-client
- Docker Compose: https://docs.docker.com/reference/cli/docker/compose/up/
- .NET Windows: https://learn.microsoft.com/es-es/dotnet/core/install/windows
- Arduino CLI: https://docs.arduino.cc/arduino-cli/installation
- Arduino IDE: https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE
- DHT Adafruit: https://github.com/adafruit/DHT-sensor-library

Los anexos del PDF incorporan íntegramente las dos guías operativas actuales, la matriz actualizada y ambos firmwares. Los documentos de cierre local anteriores se conservan como historial; este estado consolidado es el vigente para la fase.

Anexo A. Cambiar UUID y configurar la asociación en el VPS

Esta guía cambia la asociación del dispositivo con un sensor existente en la base de datos del VPS. No mueve datos de la base local, no cambia GitHub y no requiere volver a cargar el Arduino solo por cambiar el UUID.

Flujo: sensor → Arduino → USB → puente C# en Windows → HTTPS del VPS → Monitoring → dashboard.

Qué identificador cambia y dónde

Dato	Dónde se configura	Debe coincidir con
UUID del sensor	Windows: `tools/Climate.ArduinoBridge/appsettings.json`, `Device.SensorId`	VPS: `ARDUINO_SENSOR_ID` y sensor activo del catálogo
Nombre del dispositivo	Firmware: `DEVICE_ID`; Windows: `Device.DeviceId`	VPS: `ARDUINO_DEVICE_ID`
Credencial privada	Windows: `device-token.local`	SHA256 del token en `ARDUINO_TOKEN_SHA256` del VPS
Origen	JSON del firmware o generador del puente	VPS: `ARDUINO_SOURCE`
Destino	Windows: `Backend.BaseUrl`	`https://analisissistemas2026proyecto.xyz`

El UUID lo genera el backend al crear el sensor. No se escribe en el sketch: el puente hace la asociación. `device-token.local` contiene el token original; el hash de `.env` no se puede usar en su lugar.

Paso 1 — Crear o elegir el sensor en la web

1. Entra a https://analisissistemas2026proyecto.xyz con permiso para administrar sensores.
2. Abre Sensores y selecciona uno o crea otro.
3. Debe ser de tipo Temperatura, unidad `C` o `°C`, estar activo y tener comunidad asignada.
4. Guarda y abre el detalle; copia el UUID de su URL.

No confundas el sensor del VPS con un sensor de localhost. Poner un nombre como «Arduino» no basta para excluirlo del simulador del servidor: también hay que aplicar esta asociación a Monitoring.

Paso 2 — Detener el puente actual, en Windows

En su terminal pulsa `Ctrl+C`. Cierra el Monitor Serial del IDE si está abierto. No dejes dos puentes usando la misma identidad.

Si hay lecturas pendientes del UUID anterior, consérvalas. Idealmente termina de enviarlas antes de cambiar la asociación. El paso siguiente usará una cola distinta para el nuevo UUID, sin borrar la antigua ni cambiar los UUID de sus lecturas.

Paso 3 — Actualizar la asociación local, en PowerShell

Sitúate en la raíz de tu proyecto; ajusta esta ruta si lo copiaste a otra carpeta:

```
Set-Location 'C:\Users\casa\Documents\umg\desarrolloweb\webdev_proyect'
```

Elige el origen según lo que ejecutará la placa:

Modalidad	Sketch / ejecución	Origen del VPS
DHT11 físico	`backend/lecturas-sensor/lecturas-sensor.ino`	`ARDUINO`
Temperatura simulada por la placa	`hardware/arduino/temperature-sensor/temperature-sensor.ino` con SIMULATE_SENSOR=1	`SIMULATED`
Sin placa, simulación del puente	Lanzador con `-Simulate`	`SIMULATED`

Ejecuta el bloque. Cuando aparezca la pregunta, pega el UUID y pulsa Enter. El UUID no se coloca dentro del texto de `Read-Host`.

```
$newSensorId = Read-Host 'Pega el UUID del sensor del VPS y pulsa Enter'
$readingSource = Read-Host 'Escribe ARDUINO para DHT11 fisico o SIMULATED para datos de prueba'
powershell -NoProfile -ExecutionPolicy Bypass -File backend/scripts/Configure-Phase4Arduino.ps1 `
  -SensorId $newSensorId -Source $readingSource
if ($LASTEXITCODE -ne 0) { throw 'Configuracion incompleta. Corrige el error antes de continuar.' }
```

El script conserva la credencial existente; si falta, genera una nueva y habrá que registrar su hash en el VPS. No lo vuelvas a ejecutar borrando tokens para solucionar errores de red.

Confirma destino HTTPS y usa una cola por UUID:

```
$configPath = Join-Path (Get-Location) 'tools/Climate.ArduinoBridge/appsettings.json'
$config = Get-Content -LiteralPath $configPath -Raw | ConvertFrom-Json
$config.Backend.BaseUrl = 'https://analisissistemas2026proyecto.xyz'
$config.Backend.TelemetryPath = '/api/monitoring/telemetry'
$config.Backend.AllowLocalHttp = $false
$config.Buffer.Path = 'arduino-vps-' + $config.Device.SensorId + '.db'
[IO.File]::WriteAllText($configPath, ($config | ConvertTo-Json -Depth 8))
```

Si cierras PowerShell, los archivos ya guardados permanecen: no necesitas recuperar `$newSensorId` ni `$readingSource` para los pasos siguientes.

Paso 4 — Copiar solo los cinco parámetros Arduino

En PowerShell de Windows:

```
Get-Content backend/.env.phase3.local |
  Where-Object { $_ -match '^ARDUINO_(DEVICE_ID|ENABLED|SENSOR_ID|SOURCE|TOKEN_SHA256)=' }
```

Copia únicamente esas cinco líneas. No copies contraseñas de bases, JWT ni credenciales GitHub. El hash se puede registrar en el servidor; el token original debe permanecer privado en la PC.

Paso 5 — Guardar los parámetros en Bash del VPS

```
cd /opt/climate/climate_backend
nano .env.phase3.local
```

Actualiza las cinco variables con la salida de Windows. Deja una definición por variable, con `ARDUINO_ENABLED=true` y el origen elegido. Conserva el resto del archivo. Guarda con Ctrl+O, Enter; sal con Ctrl+X.

El `.env` no configura un contenedor por sí solo. Hay que inyectar las variables con el overlay y recrear Monitoring.

Paso 6 — Comprobar el overlay en el VPS

```
cd /opt/climate/climate_backend
test -f docker-compose.arduino.vps.yml && echo 'Overlay Arduino presente'
```

Si no existe, y ya se publicó en GitHub, ejecuta `git pull --ff-only`. Si aún falta, créalo con este bloque; no contiene secretos:

```
if [ ! -f docker-compose.arduino.vps.yml ]; then
cat > docker-compose.arduino.vps.yml <<'YAML'
services:
  monitoring-service:
    environment:
      Telemetry__Devices__0__DeviceId: ${ARDUINO_DEVICE_ID:?Falta ARDUINO_DEVICE_ID}
      Telemetry__Devices__0__SensorId: ${ARDUINO_SENSOR_ID:?Falta ARDUINO_SENSOR_ID}
      Telemetry__Devices__0__TokenSha256: ${ARDUINO_TOKEN_SHA256:?Falta ARDUINO_TOKEN_SHA256}
      Telemetry__Devices__0__Source: ${ARDUINO_SOURCE:-SIMULATED}
      Telemetry__Devices__0__IsActive: ${ARDUINO_ENABLED:-false}
YAML
fi
```

No uses `docker-compose.phase4.yml` local en producción: fija localhost. Tampoco ejecutes seeds para cambiar un UUID.

Paso 7 — Aplicar el cambio a Monitoring, en Bash del VPS

Este bloque recupera los archivos Compose que ya usa el contenedor y agrega el overlay solo si falta. No depende de tener la variable COMPOSE de otra terminal.

```
cd /opt/climate/climate_backend
CONFIG_FILES=$(docker inspect --format \
  '{{ index .Config.Labels "com.docker.compose.project.config_files" }}' \
  climate-phase3-vps-monitoring-service-1)
if [ -z "$CONFIG_FILES" ]; then echo 'No se encontro la configuracion de Monitoring'; else
  IFS=',' read -r -a FILES <<< "$CONFIG_FILES"
  COMPOSE=(docker compose --project-name climate-phase3-vps --env-file .env.phase3.local)
  HAS_ARDUINO=0
  for FILE in "${FILES[@]}"; do
    COMPOSE+=(-f "$FILE")
    if [ "$(basename "$FILE")" = 'docker-compose.arduino.vps.yml' ]; then HAS_ARDUINO=1; fi
  done
  if [ "$HAS_ARDUINO" -eq 0 ]; then COMPOSE+=(-f \
  /opt/climate/climate_backend/docker-compose.arduino.vps.yml); fi
  "${COMPOSE[@]}" config --quiet &&
  "${COMPOSE[@]}" up -d --no-deps --no-build --force-recreate --wait --wait-timeout 180 monitoring-service
fi
```

Para cambiar solo UUID/token/origen no hay que reconstruir código. Si la imagen del VPS todavía es de antes de Fase4, primero ejecutar `"${COMPOSE[@]}" build monitoring-service` y repetir el up solo si la construcción termina correctamente. La instalación que ya confirmó HTTP200 no necesita esa reconstrucción por cambiar el UUID.

No recrees Auth/frontend con una lista que omita sus propios overlays de dominio/OAuth. No ejecutes `down -v`.

Paso 8 — Verificar lo recibido por el contenedor

En el VPS, sin imprimir secretos:

```
docker exec climate-phase3-vps-monitoring-service-1 printenv Telemetry__Devices__0__SensorId
docker exec climate-phase3-vps-monitoring-service-1 printenv Telemetry__Devices__0__Source
docker exec climate-phase3-vps-monitoring-service-1 printenv Telemetry__Devices__0__IsActive
```

Las salidas deben coincidir con el nuevo UUID, el origen seleccionado y `true`. Desde ese momento el simulador servidor excluye ese sensor; sus históricos anteriores no se borran.

Paso 9 — Iniciar el puente en Windows

Si es la primera compilación en esa PC o cambió el código:

```
dotnet build tools/Climate.ArduinoBridge/Climate.ArduinoBridge.csproj --configuration Release
```

Después, con Monitor Serial cerrado:

```
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Climate.ArduinoBridge/Start-ArduinoBridge.ps1 `
  -PortName COM3 -Configuration Release -NoBuild
```

Cambia COM3 por el puerto real. Para leer la placa no agregues `-Simulate`; si quieres probar sin hardware, reemplaza `-PortName COM3` por `-Simulate` y usa origen SIMULATED en el VPS.

Paso 10 — Comprobar el dashboard

1. Espera `Confirmed reading` en la consola del puente: HTTP200 significa aceptación del backend.
2. En el dominio, selecciona el sensor cuyo UUID acabas de configurar.
3. Verifica temperatura, fecha, fuente e histórico.
4. Ctrl+C detiene el puente. Sin nuevas lecturas, la interfaz marca datos obsoletos al superar su umbral (30s por defecto).

Una lectura ARDUINO es física; SIMULATED puede venir del firmware de prueba o del generador .NET. El adaptador DHT11 actual persiste temperatura; humidity es solo un campo auxiliar serial.

Errores y cómo retomar

Resultado	Qué revisar
401	Token/hash/asociación activa; si hay challenge Bearer vacío, comprobar imagen/ruta pública del Gateway
403	Coincidencia exacta de UUID y source con el registro del dispositivo
400	Sensor existente y activo, tipo/unidad, valor/fecha; un POST de prueba con {} produce400 esperado
404	Imagen Monitoring/ruta Gateway antigua; recrear con --no-build no actualiza una imagen vieja
Datos cambian con puente detenido	UUID aún no asociado correctamente o existe otro emisor; comprobar paso8
COM ocupado	Cerrar Monitor Serial/otra instancia del puente
Cola de UUID anterior bloquea	No borrar ni cambiar payloads: conservar archivo antiguo y configurar cola distinta para el UUID nuevo como en paso3

Si cierras Bash, repite paso7 para recuperar COMPOSE. Si cierras Windows, vuelve a la raíz e inicia con paso9. Los archivos privados conservan configuración; no hay que repetir el seed ni regenerar credenciales. Un cambio de UUID no reasigna históricos previos.

Para revisar el punto general del proyecto:

```
Get-Content .\FASE_4_PROGRESO.md
```

Anexo B. Arduino y puente desde una PC Windows nueva

Objetivo: conectar el Arduino por USB a otra PC y enviar datos al VPS. El servidor no necesita otro despliegue si dispositivo, UUID, token y source son los mismos.

El Arduino conserva el sketch cuando se desconecta. Cambiar de PC o de puerto COM no exige volver a cargarlo. Arduino IDE es opcional si solo vas a ejecutar el puente con una placa ya programada.

Paso 1 — Detener el emisor de la PC anterior

Pulsa Ctrl+C en la consola del puente y cierra Monitor Serial. Evita dos emisores con la misma identidad. Las lecturas pendientes quedan en SQLite de la PC anterior: no copies una cola mientras su proceso está escribiendo.

No necesitas transportar el `.env` completo del backend a otra PC. Ese archivo puede contener credenciales de bases de datos, JWT y OAuth que el puente no utiliza.

Paso 2 — Copiar los archivos necesarios

Usa como carpeta de ejemplo `C:\ClimateArduino`. Puedes copiar todo el código del workspace por un medio privado, excluyendo credenciales del backend y frontend/node_modules, o copiar solamente:

```
C:\ClimateArduino\
├── tools\
│   ├── Climate.ArduinoBridge\
│   │   ├── Climate.ArduinoBridge.csproj
│   │   ├── Program.cs y todos los demas archivos .cs del proyecto
│   │   ├── Start-ArduinoBridge.ps1
│   │   ├── appsettings.example.json
│   │   ├── appsettings.json          (privado, copiar aparte)
│   │   └── device-token.local        (privado, copiar aparte)
│   └── Arduino-Uno.ps1              (si quieres compilar/cargar firmware)
├── hardware\arduino\temperature-sensor\temperature-sensor.ino
└── backend\lecturas-sensor\lecturas-sensor.ino
```

Los `.ino` solo son necesarios para recompilar/subir firmware. No copies bin/obj: se generan al compilar. Los archivos privados appsettings.json y device-token.local están ignorados por Git y no aparecen al clonar; hay que transferirlos de forma privada desde la PC que funcionaba. No pegues su contenido en chats ni los publiques en GitHub.

Con esta estructura puedes usar los mismos comandos de la guía. Si copias otra ruta, cambia el `Set-Location` correspondiente.

Si no conservas el token original, el hash del VPS no permite recuperarlo. Será necesario generar una credencial nueva y actualizar su hash en el VPS mediante la guía de cambio de asociación (GUIA_ARDUINO_CAMBIAR_SENSOR_UUID.md). No recrees credenciales si simplemente cambió el puerto COM.

Paso 3 — Instalar SDK .NET 10

En PowerShell de la nueva PC:

```
winget install --id Microsoft.DotNet.SDK.10 --exact --source winget --accept-package-agreements `
  --accept-source-agreements
```

Cierra y vuelve a abrir PowerShell después de instalar. Comprueba:

```
dotnet --list-sdks
```

Debe aparecer un SDK10.x. No basta con instalar únicamente .NET Framework o el runtime si vas a compilar. Si WinGet no existe, usa el instalador del SDK desde la guía oficial de Microsoft.

Paso 4 — Conectar el Uno y encontrar el puerto

Conecta el Arduino con un cable USB de datos. En Administrador de dispositivos de Windows abre Puertos (COM y LPT) y anota el puerto del Arduino. Puede ser COM3, COM4 u otro.

También puedes consultar desde PowerShell:

```
Get-CimInstance Win32_SerialPort | Select-Object DeviceID,Name
```

Si no aparece ningún puerto, revisa cable/USB y el controlador del modelo real de placa. Un Uno compatible puede usar un conversor USB distinto; no instales un driver CH340 para todas las placas indiscriminadamente. La PC anterior reconoció el Uno en COM3, pero la nueva puede asignar otro número.

Paso 5 — Revisar appsettings.json sin imprimir el token

```
Set-Location 'C:\ClimateArduino'
notepad tools/Climate.ArduinoBridge/appsettings.json
```

Comprueba estos datos, conservando el UUID de tu sensor registrado:

```
{
  "Arduino": {
    "PortName": "COM3",
    "BaudRate": 9600,
    "ReconnectSeconds": 5,
    "Simulate": false
  },
  "Backend": {
    "BaseUrl": "https://analisissistemas2026proyecto.xyz",
    "TelemetryPath": "/api/monitoring/telemetry",
    "TimeoutSeconds": 10,
    "AllowLocalHttp": false
  },
  "Device": {
    "DeviceId": "arduino-temp-001",
    "SensorId": "CONSERVA_EL_UUID_REAL_DEL_SENSOR_VPS"
  },
  "Buffer": {
    "Path": "arduino-vps-buffer.db",
    "MaxReadings": 10000,
    "RetentionHours": 168
  }
}
```

El texto CONSERVA_EL_UUID_REAL_DEL_SENSOR_VPS es ilustrativo: no lo guardes como valor real. Si transferiste la configuración de la PC anterior, ya debe tener el UUID correcto; no reemplaces el archivo entero por este ejemplo. El token no va dentro de este JSON: el lanzador lo obtiene de device-token.local.

Comprueba únicamente que el archivo de credencial existe:

```
Test-Path tools/Climate.ArduinoBridge/device-token.local
```

Debe responder True. Si vas a recuperar pendientes de otra PC, detén ambos puentes y conserva también los archivos SQLite/WAL/SHM del mismo conjunto antes de moverlos. No mezcles colas de dispositivos distintos. Para comenzar una cola nueva no necesitas copiar las bases de pruebas anteriores.

Paso 6 — Compilar y ejecutar el puente

Desde `C:\ClimateArduino`, con Monitor Serial cerrado:

```
dotnet build tools/Climate.ArduinoBridge/Climate.ArduinoBridge.csproj --configuration Release
```

Si termina correctamente:

```
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Climate.ArduinoBridge/Start-ArduinoBridge.ps1 `
  -PortName COM3 -Configuration Release -NoBuild
```

Cambia COM3 por el puerto encontrado. Este comando lee el Arduino físico. No agregues -Simulate si quieres leer el puerto USB; esa opción genera datos en .NET sin usar la placa.

Espera mensajes `Confirmed reading`. Abre el dashboard del dominio y selecciona el sensor correspondiente al UUID. Ctrl+C detiene el puente.

Para siguientes ejecuciones basta el comando del lanzador; no recompiles ni reinstales si el código no cambió. Windows debe permanecer encendido y con conexión a Internet para enviar. El Uno sin Wi-Fi no puede enviar HTTPS por sí mismo; si desconectas la PC, el dashboard dejará de recibir datos recientes.

Paso 7 opcional — Instalar IDE para cambiar o cargar el firmware

Omite esta sección si la placa ya tiene el programa correcto y solo cambiaste de PC.

```
winget install --id ArduinoSA.IDE.stable --exact --source winget --accept-package-agreements `
  --accept-source-agreements
```

Abre Arduino IDE:

1. En Boards Manager instala Arduino AVR Boards.
2. Para DHT11, en Library Manager instala DHT sensor library de Adafruit y acepta Adafruit Unified Sensor como dependencia.
3. Selecciona Arduino Uno y el puerto real.
4. Elige uno de los dos sketches, según la tabla siguiente.

Modo	Archivo	ARDUINO_SOURCE del VPS
DHT11 físico en DATA2	`backend/lecturas-sensor/lecturas-sensor.ino`	ARDUINO
Simulación en Uno, sin sensor	`hardware/arduino/temperature-sensor/temperature-sensor.ino`, SIMULATE_SENSOR=1	SIMULATED

Cierra el puente antes de subir. Desde PowerShell puedes usar:

```
# Compilar DHT11 sin cargar
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Arduino-Uno.ps1 -SketchPath `
  backend/lecturas-sensor
# Compilar y cargar DHT11 (reemplaza el programa actual de la placa)
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Arduino-Uno.ps1 -SketchPath `
  backend/lecturas-sensor -Upload -PortName COM3
```

O para cargar la simulación del Uno:

```
powershell -NoProfile -ExecutionPolicy Bypass -File tools/Arduino-Uno.ps1 -Upload -PortName COM3
```

No subas ambos sketches: el último reemplaza al anterior. Tras cargar, puedes comprobar JSON en Monitor Serial a9600baud y cerrarlo antes de ejecutar el puente. DHT11 físico necesita sensor cableado según las etiquetas del módulo,

DATA2 y tierra común; si emite DHT11_READ_FAILED, revisar sensor/cableado, no el dominio. La humedad aparece en JSON pero el puente actual solo persiste temperatura.

Si cambias de SIMULATED a ARDUINO, actualiza la variable ARDUINO_SOURCE del VPS y recrea Monitoring con sus overlays como explica la guía1 (GUIA_ARDUINO_CAMBIAR_SENSOR_UUID.md). No basta con cambiar el sketch.

¿Cuándo debo volver a configurar el VPS?

Cambio	¿Modificar VPS?
Otra PC con mismo Arduino, UUID, deviceId, token y source	No
Puerto COM3 cambia a COM5	No; usar -PortName COM5
Reiniciar Windows o volver a conectar el cable	No; reiniciar el puente
Nuevo UUID o token	Sí, actualizar asociación/hash y recrear Monitoring
Cambiar source SIMULATED/ARDUINO	Sí
Nuevo dispositivo independiente	Sí, registrar identidad y credencial propias; no copiar la misma identidad para dos emisores simultáneos

El overlay actual configura un dispositivo en posición0. Configurar varios dispositivos simultáneos requiere ampliar la lista Telemetry.Devices, con sensorId/deviceId únicos y credencial propia por dispositivo.

Diagnóstico y recuperación

Síntoma	Acción
No encuentra SDK	Instalar SDK10 y reabrir PowerShell
No existe token	Transferir device-token.local de forma privada; no usar el hash ni secretos OAuth
COM ocupado	Cerrar Monitor Serial y otro puente
401/403	Revisar UUID, source y token/hash; el puente detiene envíos y conserva pendientes; corregir y reiniciar
404	Endpoint/ruta/imagen del VPS desactualizados; no es un error de sensor
Red caída	Mantener SQLite; reintentos con backoff, hasta límites de buffer/retención configurados
Datos obsoletos	Revisar si el puente corre, sensor seleccionado y última lectura; el valor histórico no implica conexión actual

Al retomar, ve al paso6 si ya instalaste/copiado todo. No vuelvas a clonar ni crear otro sensor solo por reiniciar. Registra el UUID, COM y último paso completado en una nota privada, sin incluir el token. En la carpeta original puedes consultar `Get-Content .\FASE_4_PROGRESO.md`.

Referencias oficiales: .NET en Windows, instalación Arduino IDE, Arduino CLI, biblioteca DHT Adafruit.

Anexo C. Matriz completa de requisitos

Los IDs K8S/ARD son identificadores locales ligados a las secciones oficiales o del prompt; RF/RNF conservan sus IDs oficiales. No equiparar código integrado con validación de proveedores reales o hardware.

Estado actualizado al 4 de noviembre de 2026: VPS Docker/GitHub/HTTP200 reportados por el usuario; Kubernetes sigue sin ejecutarse. DHT11 compilado no equivale a medición física validada.

RF-OAUTH-01 — OAuth 2.0

ID	RF-OAUTH-01
Requisito	OAuth 2.0
Estado inicial	Sin flujo externo
En alcance	Sí
Implementación	Auth OAuthService/openid-client
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Pruebas Auth con proveedor simulado
Estado final	Implementado en código
Pendiente externo	—

RF-OAUTH-02 — OpenID Connect

ID	RF-OAUTH-02
Requisito	OpenID Connect
Estado inicial	Sin flujo externo
En alcance	Sí
Implementación	Google discovery, ID token y nonce
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Pruebas Auth con proveedor simulado
Estado final	Implementado en código
Pendiente externo	—

RF-OAUTH-03 — Google

ID	RF-OAUTH-03
Requisito	Google
Estado inicial	Sin flujo externo
En alcance	Sí
Implementación	Google OIDC + botón Angular
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Pruebas Auth con proveedor simulado
Estado final	Código integrado; login real pendiente
Pendiente externo	Credenciales reales

RF-OAUTH-04 — GitHub

ID	RF-OAUTH-04
Requisito	GitHub
Estado inicial	Sin flujo externo
En alcance	Sí
Implementación	OAuth + /user y /user/emails
Evidencia	Código, tests issuer y confirmación del usuario en VPS
Validación	Login real confirmado por usuario; correcciones Gateway/issuer/configuración
Estado final	Implementado; operación real reportada
Pendiente externo	Mantener credenciales vigentes; no se incluyen valores

RF-OAUTH-05 — Extensibilidad

ID	RF-OAUTH-05
Requisito	Extensibilidad
Estado inicial	Sin flujo externo
En alcance	Sí
Implementación	Registro de metadata revisada y adaptador OIDC
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Pruebas Auth con proveedor simulado
Estado final	Implementado en código
Pendiente externo	—

RNF-OAUTH-01 — Secretos fuera del código

ID	RNF-OAUTH-01
Requisito	Secretos fuera del código
Estado inicial	Parcial/existente
En alcance	Sí
Implementación	Variables de entorno y archivos privados ignorados
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Regresión Auth/JWT y stack local
Estado final	Implementado local; ver pruebas
Pendiente externo	—

RNF-OAUTH-03 — Administración por roles

ID	RNF-OAUTH-03
Requisito	Administración por roles
Estado inicial	Parcial/existente
En alcance	Sí
Implementación	AuthGuard/guards/interceptor y policies existentes
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Regresión Auth/JWT y stack local
Estado final	Implementado local; ver pruebas
Pendiente externo	—

RNF-OAUTH-04 — Rechazar JWT expirado

ID	RNF-OAUTH-04
Requisito	Rechazar JWT expirado
Estado inicial	Parcial/existente
En alcance	Sí
Implementación	Tokens.verify y validadores por servicio
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Regresión Auth/JWT y stack local
Estado final	Implementado local; ver pruebas
Pendiente externo	—

RNF-OAUTH-05 — Validación de APIs internas

ID	RNF-OAUTH-05
Requisito	Validación de APIs internas
Estado inicial	Parcial/existente
En alcance	Sí
Implementación	JWT + claves internas restringidas existentes
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Regresión Auth/JWT y stack local
Estado final	Implementado local; ver pruebas
Pendiente externo	—

RNF-OAUTH-02 — Secrets Kubernetes reales

ID	RNF-OAUTH-02
Requisito	Secrets Kubernetes reales
Estado inicial	Manifiestos de ejemplo
En alcance	No ejecutar
Implementación	Inyección por configuración preparada
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Validación productiva no realizada
Estado final	Pendiente externo
Pendiente externo	Secret store y responsable de infraestructura

K8S-10-CLUSTER — Clúster y K3s

ID	K8S-10-CLUSTER
Requisito	Clúster y K3s
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-11-PODS — Pods

ID	K8S-11-PODS
Requisito	Pods
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-11-DEPLOY — Deployments

ID	K8S-11-DEPLOY
Requisito	Deployments
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-12-SERVICE — Services internos

ID	K8S-12-SERVICE
Requisito	Services internos
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-12-INGRESS — Ingress HTTPS

ID	K8S-12-INGRESS
Requisito	Ingress HTTPS
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-13-CONFIG — ConfigMaps

ID	K8S-13-CONFIG
Requisito	ConfigMaps
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-13-SECRET — Secrets productivos

ID	K8S-13-SECRET
Requisito	Secrets productivos
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-14-PV — Persistent Volumes/PVC

ID	K8S-14-PV
Requisito	Persistent Volumes/PVC
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-14-HEALTH — Probes liveness/readiness

ID	K8S-14-HEALTH
Requisito	Probes liveness/readiness
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-OBJ7-SCALE — Escalabilidad horizontal

ID	K8S-OBJ7-SCALE
Requisito	Escalabilidad horizontal
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-OBJ8-DNS — Dominio DNS

ID	K8S-OBJ8-DNS
Requisito	Dominio DNS
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

K8S-OBJ8-TLS — TLS público

ID	K8S-OBJ8-TLS
Requisito	TLS público
Estado inicial	Manifiestos/documentación Fase3
En alcance	No ejecutar
Implementación	backend/k8s reutilizable
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	No aplicado ni validado en VPS
Estado final	Excluido; no cumplido
Pendiente externo	Ver FASE_4_PENDIENTES_FUERA_SCOPE.md

F3-CONTINUIDAD — Servicios heterogéneos, bases, reglas, reportes, auditoría y simulador

ID	F3-CONTINUIDAD
Requisito	Servicios heterogéneos, bases, reglas, reportes, auditoría y simulador
Estado inicial	Fase3 implementada
En alcance	Sí preservar
Implementación	No reemplazar servicios propietarios
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Regresión + stack Docker local
Estado final	Implementado local; ver pruebas
Pendiente externo	—

ARD-08 — USB serial sin asumir conectividad

ID	ARD-08
Requisito	USB serial sin asumir conectividad
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	ISerialDataSource/System.IO.Ports
Evidencia	Tests locales y HTTP200 aportado por usuario del VPS
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; aceptación de telemetría en VPS reportada
Pendiente externo	—

ARD-09 — Arquitectura PC → Gateway → Monitoring existente

ID	ARD-09
Requisito	Arquitectura PC → Gateway → Monitoring existente
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Sin microservicio ni Hub adicional
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-10 — Firmware LM35 y generador simulado

ID	ARD-10
Requisito	Firmware LM35 y generador simulado
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Sketch simulado/LM35 y adaptador DHT11; compilados para Uno
Evidencia	Arduino CLI y salida upload aportada por usuario
Validación	Simulado: flash3920/RAM204; DHT11: flash5626/RAM243
Estado final	Compilados; carga del simulador confirmada; medición DHT11 pendiente
Pendiente externo	Confirmar carga/medición/calibración DHT11

ARD-11-PORT — Puerto/baud/detección/reconexión

ID	ARD-11-PORT
Requisito	Puerto/baud/detección/reconexión
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	SerialDataSource + AcquisitionWorker
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-11-DTO — DTO tipado/validación/asociación

ID	ARD-11-DTO
Requisito	DTO tipado/validación/asociación
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Protocol.Parse + DeviceOptions
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-11-HTTP — HttpClient/DI/Worker/cancelación

ID	ARD-11-HTTP
Requisito	HttpClient/DI/Worker/cancelación
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	TransmissionWorker y configuración validada
Evidencia	Tests locales y HTTP200 aportado por usuario del VPS
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; aceptación de telemetría en VPS reportada
Pendiente externo	—

ARD-12 — Opciones, secretos y HTTPS

ID	ARD-12
Requisito	Opciones, secretos y HTTPS
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	appsettings.example y token externo; loopback explícito
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-13-BUFFER — Cola persistente acotada

ID	ARD-13-BUFFER
Requisito	Cola persistente acotada
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	SQLite WAL, MaxReadings, FIFO y retención
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-13-RETRY — Backoff y errores permanentes

ID	ARD-13-RETRY
Requisito	Backoff y errores permanentes
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	429/5xx reintentan; 400 elimina con diagnóstico; 401/403 detiene
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-13-ID — Idempotencia estable

ID	ARD-13-ID
Requisito	Idempotencia estable
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	UUID en payload persistido antes de HTTP
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-14 — Ingestión limitada por dispositivo

ID	ARD-14
Requisito	Ingestión limitada por dispositivo
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	TelemetryController + hash/token + catálogo
Evidencia	Tests locales y HTTP200 aportado por usuario del VPS
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; aceptación de telemetría en VPS reportada
Pendiente externo	—

ARD-15 — Dispositivo/sensor/comunidad/tipo/estado/fuente

ID	ARD-15
Requisito	Dispositivo/sensor/comunidad/tipo/estado/fuente
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Configuración por sensor; comunidad del catálogo existente
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-16 — Reglas existentes y ausencia de alertas duplicadas

ID	ARD-16
Requisito	Reglas existentes y ausencia de alertas duplicadas
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Dispatch persistente + readingId procesado por Java
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-17 — SignalR y recuperación REST

ID	ARD-17
Requisito	SignalR y recuperación REST
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Hub existente, eventos y sincronización REST
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-18 — Gráfico actual/histórico y selección de sensor

ID	ARD-18
Requisito	Gráfico actual/histórico y selección de sensor
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	TemperaturePanel usa SensorChart existente sin gráfica agregada duplicada
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-19 — Simulación convive y fuentes distinguibles

ID	ARD-19
Requisito	Simulación convive y fuentes distinguibles
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Exclusión del sensor de dispositivo; Source en SQL/DTO/chart
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-20 — Pruebas sin hardware

ID	ARD-20
Requisito	Pruebas sin hardware
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Pruebas .NET, Auth, Angular e integración Docker
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-21 — UX: accesibilidad, loading/error/empty y datos obsoletos

ID	ARD-21
Requisito	UX: accesibilidad, loading/error/empty y datos obsoletos
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Panel responsive, etiquetas, estado por antigüedad y foco visible
Evidencia	Código y docs/FASE_4_PRUEBAS.md
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

ARD-24 — Documentación y matriz

ID	ARD-24
Requisito	Documentación y matriz
Estado inicial	No existía integración Arduino
En alcance	Sí
Implementación	Markdown, guías UUID/nuevaPC, PDF Fase4 y PDF general actualizado
Evidencia	docs/FASE_4_DOCUMENTACION_COMPLETA.md y PDFs generados
Validación	Ver FASE_4_PRUEBAS.md
Estado final	Implementado; evidencia en informe
Pendiente externo	—

Anexo D. Firmware DHT11 físico

Código fuente incluido para consulta. Compilado para Arduino Uno; la temperatura utiliza source ARDUINO. Copiar/cargar desde el archivo .ino original para conservar el texto exacto.

```
#include <Arduino.h>
#include <DHT.h>

constexpr uint8_t DHT_PIN = 2;
constexpr uint8_t DHT_TYPE = DHT11;
constexpr unsigned long READ_INTERVAL_MS = 3000;
constexpr char DEVICE_ID[] = "arduino-temp-001";

DHT dht(DHT_PIN, DHT_TYPE);
unsigned long lastReadMs = 0;
unsigned long sequenceNumber = 0;

void setup() {
  Serial.begin(9600);
  dht.begin();

  // Permite realizar la primera lectura inmediatamente.
  lastReadMs = millis() - READ_INTERVAL_MS;
}

void loop() {
  unsigned long currentMs = millis();

  if (currentMs - lastReadMs < READ_INTERVAL_MS) {
    return;
  }

  lastReadMs = currentMs;

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();
```

Código (continuación)

```
  if (isnan(temperature) || isnan(humidity) || temperature < -55 || temperature > 150 || humidity < 0 ||
  humidity > 100) {
    Serial.print(F("{\"protocolVersion\":1,\"deviceId\":\""));
    Serial.print(DEVICE_ID);
    Serial.println(F("\",\"error\":\"DHT11_READ_FAILED\"}"));
    return;
  }

  // Una trama por linea, compatible con Climate.ArduinoBridge.
  // Sin reloj UTC en la placa: el puente asigna la fecha de recepcion.
  Serial.print(F("{\"protocolVersion\":1,\"deviceId\":\""));
  Serial.print(DEVICE_ID);
  Serial.print(F("\",\"sensorType\":\"TEMPERATURE\",\"value\":"));
  Serial.print(temperature, 2);
  Serial.print(F(",\"unit\":\"C\",\"source\":\"ARDUINO\",\"sequence\":"));
  Serial.print(sequenceNumber++);
  // Dato auxiliar visible en Monitor Serial; el puente actual solo persiste temperatura.
  Serial.print(F(",\"sensor\":\"DHT11\",\"humidity\":"));
  Serial.print(humidity, 2);
  Serial.println(F("}"));
}
```

Anexo E. Firmware simulado y referencia LM35

SIMULATE_SENSOR=1 produce datos de prueba sin sensor físico. Con valor0 se requiere el cableado y referencia ADC del LM35.

```
// Referencia: Arduino Uno, LM35 en A0 (VCC 5V, GND común).
// SIMULATE_SENSOR=1 genera datos de prueba sin conectar sensores.
#define SIMULATE_SENSOR 1
#define SENSOR_PIN A0
#define SERIAL_BAUD 9600
#define SAMPLE_INTERVAL_MS 2000UL
#define ADC_REFERENCE_VOLTS 5.0
#define DEVICE_ID "arduino-temp-001"

unsigned long previousSample = 0;
unsigned long sequenceNumber = 0;

void setup() {
  Serial.begin(SERIAL_BAUD);
  pinMode(SENSOR_PIN, INPUT);
}

void loop() {
  unsigned long now = millis();
  if (now - previousSample < SAMPLE_INTERVAL_MS) return;
  previousSample = now;
  float temperature;
#if SIMULATE_SENSOR
  // Oscila entre 20 y 40 °C para probar también las reglas de alerta.
  temperature = 30.0 + 10.0 * sin(sequenceNumber * 0.15);
#else
  temperature = analogRead(SENSOR_PIN) * (ADC_REFERENCE_VOLTS / 1023.0) * 100.0;
#endif
  if (isnan(temperature) || temperature < -55 || temperature > 150) {
    Serial.println(F("{\"protocolVersion\":1,\"deviceId\":\"" DEVICE_ID
  "\",\"error\":\"INVALID_TEMPERATURE\"}"));
    return;
```

Código (continuación)

```
  }
  Serial.print(F("{\"protocolVersion\":1,\"deviceId\":\"" DEVICE_ID
  "\",\"sensorType\":\"TEMPERATURE\",\"value\":"));
  Serial.print(temperature, 2);
  Serial.print(F(",\"unit\":\"C\",\"source\":\""));
#if SIMULATE_SENSOR
  Serial.print(F("SIMULATED"));
#else
  Serial.print(F("ARDUINO"));
#endif
  Serial.print(F("\",\"sequence\":"));
  Serial.print(sequenceNumber++);
  Serial.println(F("}"));
}
```

Anexo F. Regeneración y evidencias

Los comandos largos del PDF pueden mostrarse en varias líneas con continuaciones de shell. El Markdown fuente conserva los comandos completos editables. No leer ni publicar archivos privados para regenerar.

```
python -m pip install --target tools/pdf-libs -r tools/requirements-phase4-pdf.txt
python tools/phase4-doc-matrix.py
python tools/generate-phase4-pdf.py
```

El PDF general preserva sus páginas previas desde docs/archive/Documentacion_Climate_Monitoring_System_antes_Fase_4.pdf y agrega esta edición una sola vez. Volver a ejecutar no duplica el anexo. Para revisar avance: Get-Content .\FASE_4_PROGRESO.md.

Documentación técnica consolidada — Fase 3

Climate Monitoring System
Entrega de Fase 3 — 21 de octubre de 2026

Arquitectura, funcionalidades, contratos, operación y validación del sistema.

Contenido de la entrega

- FASE 3 DOCUMENTACION CONSOLIDADA

Participación:

Christopher Herrera 100

Erwin Ramirez 100

Cinthia Robles 100

Climate Monitoring System — Documentación consolidada de Fase 3

Fecha: 21 de octubre de 2026. Este documento describe la implementación del sistema, sus funcionalidades, contratos, componentes y procedimientos de operación. Reúne frontend, backend, persistencia, comunicación y resultados de validación.

1. Visión general

El sistema registra mediciones climáticas por sensor y comunidad, simula estaciones, evalúa riesgos, administra la atención de alertas, conserva eventos, registra auditoría y genera ocho tipos de informes PDF. La interfaz Angular consume un Gateway único. Cada microservicio es propietario de sus datos y expone contratos HTTP para que otros servicios consulten su dominio.

Los componentes activos son Frontend, Gateway, Auth, Community, Sensor, Monitoring, Alert, Event, Audit y Report. Docker Compose ejecuta 17 contenedores: diez componentes de aplicación, tres PostgreSQL, dos MongoDB, SQL Server y RabbitMQ. Los servicios .NET anteriores de Identity, Alert y Audit se conservan como código de referencia; los proveedores activos de esos dominios son NestJS, Spring Boot y FastAPI respectivamente.

2. Mapa de tecnologías y microservicios

Componente	Lenguaje y framework	Persistencia	Responsabilidad	Ubicación en backend, salvo frontend
Frontend	TypeScript, Angular 22.2.3, TypeScript 6.0.3	Estado de interfaz	Pantallas, formularios, gráficos, permisos visuales y reportes	frontend/climate-monitoring-web/ src/app
Gateway	C#, ASP.NET Core .NET 10, YARP	Sin base propia	Enrutamiento, JWT, CORS, límites y WebSocket	src/Gateway/ Climate.Gateway
Auth	TypeScript, NestJS, Node.js	PostgreSQL exclusivo	Usuarios, roles, login, JWT, logout e identidad de servicio	src/Services/ AuthService
Community	C#, ASP.NET Core .NET 10	PostgreSQL exclusivo	Catálogo de comunidades, estados, ubicación y filtros	src/Services/ CommunityServic e/ Climate.Communi ties.Api
Sensor	C#, ASP.NET Core .NET 10	SensorDb en SQL Server	Sensores, tipos, unidades, ubicación y referencias a comunidades	src/Services/ SensorService
Monitoring	C#, ASP.NET Core .NET 10, SignalR	MonitoringDb en SQL Server	Lecturas, simulación, dashboard y actualización en vivo	src/Services/ MonitoringService
Alert	Java 21, Spring Boot 3.5.16, JDBC	PostgreSQL exclusivo	Reglas, evaluación de riesgos, alertas y atención	src/Services/ AlertServiceJava
Event	C#, ASP.NET Core .NET 10	EventDb en SQL Server	Historial de eventos climáticos y estadísticas	src/Services/ EventService

Audit	Python, FastAPI	MongoDB exclusivo	Consumo de auditoría, deduplicación e historial	src/Services/ AuditServicePython
Report	Python, FastAPI, Jinja2, WeasyPrint	MongoDB exclusivo y volumen PDF	Trabajos, consultas HTTP, generación, historial y descarga	src/Services/ ReportService
Mensajería	RabbitMQ	Volumen de mensajes	Entrega asíncrona de auditoría desde Outbox	Configuración Docker Compose

Las bibliotecas Python compartidas de seguridad y publicación están en src/Services/PythonCommon. El frontend utiliza Signals, componentes standalone, rutas lazy, RxJS, Chart.js, Tailwind CSS y SCSS.

3. Mapeo de rutas públicas

Ruta en Gateway	Servicio destino	Función
/api/auth y /api/users	Auth NestJS	Autenticación y administración de usuarios
/api/communities	Community .NET	Comunidades y sus estados
/api/sensors	Sensor .NET	Catálogo y administración de sensores
/api/monitoring	Monitoring .NET	Lecturas y simulación
/api/dashboard	Monitoring .NET	Resumen agregado para la interfaz
/api/alerts y /api/alert-rules	Alert Java	Alertas, atención y reglas
/api/events	Event .NET	Historial y estadísticas de fenómenos
/api/audit	Audit FastAPI	Consulta de auditoría
/api/reports	Report FastAPI	Generación, historial y descarga PDF

Gateway transforma las rutas REST públicas al prefijo interno /api/v1 de cada dominio. También transporta la conexión WebSocket de SignalR. Las APIs internas de servicio se protegen mediante clave interna y no se publican como rutas de navegador. El contrato público OpenAPI está en backend/docs/openapi/climate-api-v1.json.

4. Propiedad de datos e integración

Auth, Community y Alert usan PostgreSQL independientes; Audit y Report usan MongoDB independientes. Sensor, Monitoring y Event mantienen bases SQL Server separadas. Compartir motor no implica compartir tablas ni responsabilidad de dominio.

Community es propietario del catálogo de comunidades. Sensor valida las referencias consultando Community por HTTP y mantiene snapshots en memoria para la consulta; no crea una segunda copia persistente del catálogo. Community obtiene los conteos de sensores mediante la API interna de Sensor. La migración EF de Sensor retira la antigua relación de clave foránea local para permitir esta separación.

Report obtiene información mediante las APIs de Community, Sensor, Monitoring, Alert y Event. Su MongoDB almacena parámetros, trabajos, metadatos y descargas. Las consultas de informes no acceden directamente a las bases de otros servicios.

Los modelos intercambian UUID, fechas con zona, estados y tipos de sensor compatibles entre C#, TypeScript, Java y Python. El Gateway conserva el contrato del cliente durante el cambio de implementación de los servicios.

5. Autenticación, roles y seguridad

Auth implementa login con usuario y contraseña, administración de usuarios, roles y desactivación. La respuesta de autenticación conserva accessToken, expiresAt y user. Se admite la verificación de hashes ASP.NET Identity PBKDF2 V2/V3 para compatibilidad con usuarios transferidos.

Los JWT usan firma HS256, issuer ClimateMonitoring.Identity y audience ClimateMonitoring.Client. Los validadores comprueban firma, expiración, emisor y audiencia. Se incluyen roles y permisos explícitos junto con el claim de rol compatible con .NET. Logout registra auditoría, elimina la sesión del cliente y Auth rechaza el identificador de token cerrado en sus propios endpoints; los demás servicios aplican la expiración del JWT.

Rol	Generar reportes	Consultar historial	Descargar PDF
Administrator	Sí	Todos	Sí
Operator	Sí	Propios	No
Viewer	No	Propios	No

Los permisos reports:generate, reports:read y reports:download se verifican en backend. El navegador no elige autor, archivo físico ni URL de dependencia. Para consultar dominios, el worker de Report obtiene de Auth una identidad Viewer de servicio con vida de cinco minutos mediante un endpoint interno protegido.

Los secretos se suministran por entorno. Los archivos .env privados y las contraseñas no se incluyen en la documentación. Gateway agrega CorrelationId y aplica controles de acceso y límites de solicitudes. Los PDF usan almacenamiento privado y recursos de renderizado restringidos.

6. Comunidades y sensores

Las comunidades incluyen nombre, descripción, coordenadas, municipio, departamento, país, estado y fecha de creación. La API admite filtros, activación, desactivación y consultas paginadas. Los sensores incluyen código, nombre, descripción, tipo, unidad, comunidad y coordenadas. Se validan códigos, referencias y rangos geográficos antes de guardar.

La desactivación preserva el registro y sus referencias históricas. El catálogo activo determina los sensores procesados por la simulación. La interfaz conserva formularios, confirmaciones, mensajes de error y controles según el rol.

7. Lecturas y simulación

Monitoring registra lecturas asociadas a sensor y comunidad, con tipo, valor, unidad y fecha. Expone lecturas actuales, última lectura por sensor, historial, gráficos y búsqueda paginada por comunidad, sensor, tipo y rango temporal.

El simulador utiliza el catálogo de sensores activos y genera lotes periódicos. Se dispone de acciones para iniciar, detener y reiniciar la simulación y de valores de simulación por sensor. La configuración local utiliza un intervalo de 30 segundos. Las nuevas mediciones recorren las mismas APIs y evaluación de riesgos que las lecturas registradas manualmente.

La demostración inicial contiene Ciudad de Guatemala, Puerto Barrios y Quetzaltenango, con cinco sensores por comunidad: temperatura, humedad, velocidad de viento, lluvia y nivel de agua. Se cargan 27 reglas y primeras mediciones para sensores sin datos. La carga busca comunidades por nombre, sensores por código y reglas por nombre, conserva cambios existentes y se puede repetir sin duplicar catálogos.

8. Reglas, alertas y eventos

Alert Java administra umbrales mínimos y máximos por tipo de sensor, fenómeno y nivel. Los fenómenos son inundación, sequía, tormenta, helada e incendio forestal. Las reglas iniciales distribuyen umbrales Yellow, Orange y Red para nueve combinaciones de tipo y fenómeno.

La evaluación selecciona la mayor gravedad por fenómeno, evita duplicaciones por lectura y mantiene las transiciones de alerta activa, atendida y cerrada. Conserva responsables, fechas y snapshots de la regla que originó la alerta. PostgreSQL utiliza índices, bloqueos por fila y advisory locks para coordinar las operaciones concurrentes.

Flujo de medición: Sensor proporciona el contexto; Monitoring almacena la lectura; Alert evalúa reglas; Event registra el historial; Monitoring publica actualizaciones SignalR; los productores envían auditoría mediante Outbox y RabbitMQ. La entrega hacia Event y SignalR se registra de forma durable después del commit y admite reintentos.

9. Auditoría y mensajería

Las operaciones de negocio registran su evento Outbox en la misma transacción de persistencia. Un publicador independiente entrega el mensaje con confirmación RabbitMQ. La caída del broker o del consumidor no incorpora una llamada de auditoría remota dentro de la transacción de negocio.

Audit FastAPI consume eventos, deduplica por EventId y conserva el historial en MongoDB. El contrato incluye nombres de evento compatibles, SchemaVersion y CorrelationId. Se utilizan confirmaciones, reintentos y tratamiento de mensajes rechazados. La interfaz consulta el historial mediante paginación.

10. Dashboard y experiencia de usuario

El dashboard muestra comunidades, sensores activos e inactivos, lecturas actuales, cantidad de alertas activas y distribución por nivel. Incorpora eventos por fenómeno, evolución y estadísticas de las últimas 24 horas agrupadas por tipo y unidad: cantidad, promedio, mínimo y máximo. El filtro de comunidad se aplica a las consultas correspondientes.

SignalR actualiza la información y la recuperación REST reconstruye el estado al reconectar. La interfaz diferencia carga, ausencia de resultados, error y recuperación. Se mantienen navegación por rol, diseño móvil, foco de diálogos, recorrido de teclado, Escape, restitución de foco y validaciones asociadas a sus campos.

11. Ocho tipos de reportes PDF

Tipo	APIs de origen	Contenido
COMMUNITIES	Community, Sensor	Ubicación, estado y cantidad de sensores
SENSORS	Sensor, Community	Identificación, tipo, unidad, comunidad y estado
READINGS	Monitoring y catálogos	Mediciones con fechas, sensor, comunidad y unidades
ALERTS	Alert y catálogos	Fenómeno, nivel, valor, umbral, mensaje y atención
EVENTS	Event y catálogos	Historial de fenómenos y estados
SENSOR_STATUS	Sensor, Monitoring	Actividad, comunicación y última lectura
CLIMATE_STATISTICS	Monitoring, Alert	Cantidades, promedios, extremos y alertas del periodo
EXECUTIVE	APIs de dominio	Cobertura, indicadores, niveles y conclusiones cuantificadas

Los filtros admiten sensor, comunidad, tipo, municipio, departamento, búsqueda, actividad, fenómeno, estado y fechas según el informe. READINGS y CLIMATE_STATISTICS requieren rango temporal. EXECUTIVE utiliza las últimas 24 horas si no se especifican fechas. El rango máximo configurable es de 366 días. Las fechas incluyen zona horaria.

Las agregaciones utilizan Decimal y separan tipos y unidades. La lluvia se acumula cuando la configuración confirma mediciones incrementales en milímetros; otras semánticas se presentan sin inventar acumulados. El estado sin comunicación se calcula con un umbral predeterminado de 900 segundos, mientras que un sensor desactivado se identifica como inactivo.

12. API y ciclo de generación

Método y ruta	Respuesta
GET /api/reports/types	Catálogo de los ocho tipos
POST /api/reports/generate	HTTP 202, identificador y estado inicial del trabajo
GET /api/reports	Historial con items, total, page y pageSize
GET /api/reports/statistics	Totales y distribución por estado con control de usuario
GET /api/reports/{id}	Metadatos y progreso
GET /api/reports/{id}/download	Archivo PDF autenticado con nombre de descarga

El trabajo pasa por cola, procesamiento y finalización correcta o fallida. MongoDB conserva usuario, parámetros, fechas, progreso, error, tamaño y SHA-256. El worker reclama el trabajo atómicamente, renueva un lease de 60 segundos cada 15 segundos y recupera ejecuciones interrumpidas hasta tres intentos. La UI consulta el estado cada cinco segundos y presenta historial paginado de 25 entradas por página.

Las consultas de datos usan páginas de 250 registros. El límite predeterminado es de 50 000 filas y el exceso produce un error explícito. WeasyPrint procesa HTML Jinja2 en un subproceso con tiempo máximo de 180 segundos y tamaño máximo de PDF de 25 MiB. El diseño utiliza A4 horizontal, identificación CM, filtros, usuario, fecha, zona horaria, encabezados repetidos y numeración.

La concurrencia por usuario se protege mediante índice MongoDB único sobre el cupo activo. En la prueba de 20 solicitudes simultáneas se aceptaron tres y se respondieron 17 con HTTP 429. Los cupos se liberan al finalizar, fallar o agotar intentos.

Los archivos se identifican mediante UUID del servidor y permanecen en report-files. La retención predeterminada es de 30 días; el historial persiste después de la expiración. Descargar antes de completar responde 409 y descargar un archivo expirado responde 410. Se registra cada descarga y se auditan generación y descarga mediante publicación durable.

13. Organización del frontend

La funcionalidad de reportes se ubica en src/app/features/reports, con servicio HTTP tipado, formulario, historial, detalle y descarga. Las rutas se registran en src/app/app.routes.ts y el menú en src/app/core/layout/navigation.model.ts. Dashboard y Audit adaptan sus modelos y consultas a los contratos actuales.

La aplicación usa componentes standalone, Signals y OnPush, formularios validados y rutas lazy. La configuración /runtime-config.json define el Gateway público para REST y SignalR. El contenedor frontend sirve Angular y su configuración pública; las bases y endpoints internos se consultan desde backend.

14. Ejecución y configuración local

Desde backend:

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/Start-Phase3.ps1 -NoBuild
```

El comando valida Compose, inicia los servicios, espera sus healthchecks, carga la demostración y activa la simulación. Omitir -NoBuild construye las imágenes. Frontend: http://localhost:4200. Gateway: http://localhost:8080. El proyecto Docker es climate-phase3-local.

Initialize-Phase3.ps1 crea .env.phase3.local cuando no existe y preserva las credenciales existentes. El usuario inicial es admin y su contraseña se consulta en AUTH_ADMIN_SEED_PASSWORD del archivo privado. PHASE3_DEMO_ENABLED=false permite iniciar sin carga automática. Seed-Phase3Demo.ps1 completa los datos iniciales cuando los servicios ya están ejecutándose.

El Compose predeterminado contiene la arquitectura completa y los overlays docker-compose.phase3*.yml son su fuente de configuración. scripts/update-phase3-compose.ps1 regenera el archivo. Los nombres de volúmenes respetan el proyecto Docker, separando instalaciones y pruebas. Se construyeron diez imágenes de aplicación climate/*:phase3.

15. Respaldo y recuperación

Se respaldan ocho volúmenes de datos: SQL Server, RabbitMQ, tres PostgreSQL, dos MongoDB y archivos PDF. El procedimiento validado realiza snapshots con servicios detenidos, verifica SHA-256 y restaura en otro proyecto. Report MongoDB y report-files se recuperan conjuntamente para conservar la correspondencia entre metadatos y archivos.

La recuperación comprobó login, comunidades, sensores, lecturas, reglas, alertas, eventos, auditoría, historial y descarga PDF con hash idéntico. Las herramientas de transferencia SQL hacia PostgreSQL y MongoDB verifican UUID, campos, conteos y fingerprints, admiten repetición y rechazan colisiones sin sobrescribir el origen.

16. Configuración de infraestructura

Los manifiestos Kubernetes incluyen servicios, probes, recursos, secretos y almacenamiento PDF de 5 GiB con estrategia Recreate y acceso RWO. Se validaron 29 recursos y un Secret mediante esquema. Las bases se configuran como dependencias privadas. La operación con varias instancias de Monitoring requiere coordinación del simulador y distribución de SignalR mediante backplane o afinidad.

La configuración conserva interoperabilidad JWT HS256 y una tabla de identidades externas para integrar proveedores de identidad. Los archivos de infraestructura y las guías de configuración forman parte del repositorio técnico.

17. Resultados de validación

Área	Resultado registrado
.NET	122 pruebas aprobadas y una omitida
Alert Java	Siete pruebas aprobadas con integración PostgreSQL
Auth	Cinco pruebas aprobadas
Servicios Python	Once pruebas aprobadas
Angular	56 pruebas aprobadas, lint y build correctos
Accesibilidad	Seis escaneos aislados y 16 sobre Compose real sin violaciones detectadas en los recorridos evaluados
Reporte de volumen	5 000 lecturas, 20 páginas de API, PDF de 1 061 586 bytes en 38,312 segundos
Concurrencia	Tres trabajos aceptados y 17 respuestas 429 entre 20 solicitudes simultáneas
Demostración	Tres comunidades, 15 sensores, 27 reglas; lecturas de 45 a 60 y seed sin duplicados
Operación	17 contenedores saludables y recuperación

	verificada de ocho volúmenes

La integración verifica JWT, roles, filtros, generación de los ocho PDF, MIME, tamaño, hash, expiración, fallos de dependencias, recuperación de trabajos y continuidad del negocio ante fallos RabbitMQ. Los recorridos de navegador cubren login/logout, navegación, reglas, formularios, móvil, teclado y descarga.

Los resultados describen las ejecuciones realizadas y su entorno de prueba. Los escaneos de accesibilidad documentan los recorridos evaluados y las mediciones de rendimiento corresponden al equipo y datos utilizados.

18. Referencias técnicas de mantenimiento

Contratos: backend/docs/openapi/climate-api-v1.json. Pruebas: backend/docs/FASE_3_PRUEBAS.md. Evidencias: backend/docs/validation/phase3-demo.json, phase3-large-report.json, phase3-report-integration.json y phase3-recovery.json.

Scripts de validación: Test-Phase3Auth.ps1, Test-Phase3Alerts.ps1, Test-Phase3Audit.ps1, Test-Phase3System.ps1, Test-Phase3Compose.ps1, Test-Phase3Recovery.ps1 y Test-Phase3Demo.ps1. Frontend: scripts/test-browser.mjs, scripts/test-reports-browser.mjs, npm run test:ci, npm run lint y npm run build.

El generador backend/scripts/generate-phase3-word.py --consolidated-only reconstruye este Word desde la fuente Markdown consolidada. Requiere Python y python-docx. El documento reúne la arquitectura vigente y los comportamientos implementados, con referencias a código y evidencia para facilitar su exposición y mantenimiento.